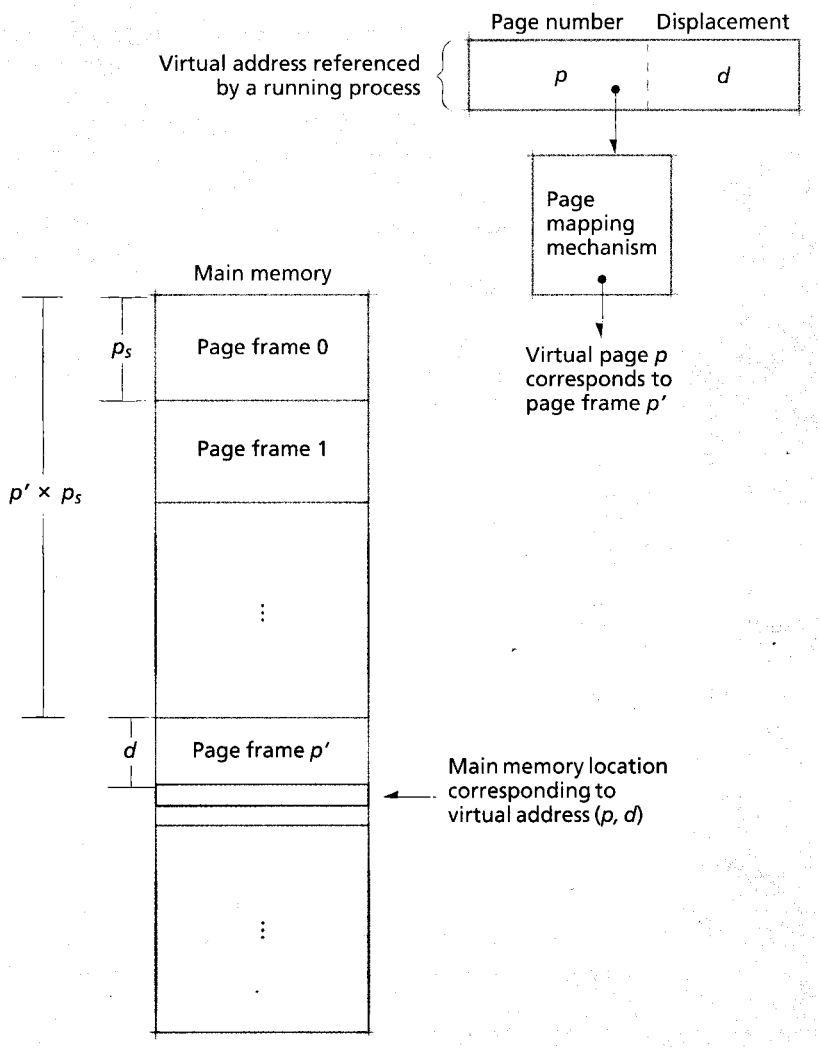


Figure 10.10 *| Correspondence between virtual memory addresses and physical memory addresses in a pure paging system.*

ory address and places *d* in the least-significant bits of the real memory address. For example, consider a 32-bit system *(n* = 32) using 4KB pages *(m* - 12). Each page frame number is represented using 20 bits, so that page frame number 15 would be represented as the binary string 00000000000000001111. Similarly, a displacement of 200 would be represented as the 12-bit binary string 000011001000. The 32-bit address of this location in main memory is formed simply by concatenating the two strings, yielding 00000000000000001111000011001000. This demonstrates how

paging simplifies block mapping by using the concatenation operation to form a real memory address. On the contrary, with variable-size blocks, the system must perform an addition operation to form the real memory address from the ordered pair $r = (b',d)$.

Now let us consider dynamic address translation in more detail. One benefit of a paged virtual memory system is that not all pages belonging to a process must reside in main memory at the same time—main memory must contain only the page (or pages) in which the process is currently referencing an address (or addresses). The advantage is that more processes may reside in main memory at the same time. The disadvantage comes in the form of increased complexity in the address translation mechanism. Because main memory normally does not contain all of a process's pages at once, the page table must indicate whether or not a mapped page currently resides in main memory. If it does, the page table indicates the number of the page frame in which it resides. Otherwise, the page table yields the location in secondary storage at which the referenced page can be found. When a process references a page that is not in main memory, the processor generates a **page fault,** which invokes the operating system to load the missing page into memory from secondary storage.

Figure 10.11 shows a typical **page table entry (PTE).** A resident bit, *r,* is set to 0 if the page is not in main memory and 1 if it is. If the page is not in main memory, then s is its secondary storage address. If the page is in main memory, then p' is its frame number. Over the next several subsections we consider several implementations of the page mapping mechanism.

Self Review

1. Does the page mapping mechanism require that s and p' be stored in separate cells of a PTE, as shown in Fig. 10.11?
2. Compare and contrast the notions of a page and a page frame.

Ans: **1)** No. To reduce the amount of memory consumed by PTEs, many systems contain only one cell to store either the page frame number or the secondary storage address. If the

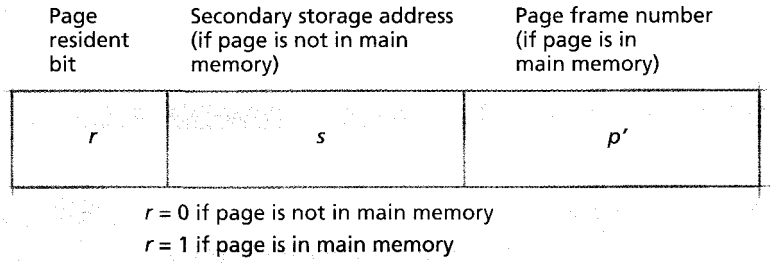


Figure 10.11 | *Page table entry.*

resident bit is on. the PTE stores a page frame number, but not a secondary storage address. If the resident bit is off, the PTE stores a secondary storage address, but not a page frame number. **2)** Pages and page frames are identical in size; a page refers to a fixed-size block of a process's virtual memory space and a page frame refers to a fixed-size block of main memory. A virtual memory page must be loaded into a page frame in main memory before a processor can access its contents.

10.4.1 Paging Address Translation by Direct Mapping

In this section we consider the **direct-mapping** technique for translating virtual addresses to physical addresses in a pure paging system. Direct mapping proceeds as illustrated in Fig. 10.12.

A process references virtual address v = *(p, d).* At context-switch time the operating system loads the main memory address of a process's page table into the **page table origin register.** To determine the main memory address that corresponds to the referenced virtual address, the system first adds the process's page table base address, *b,* to the referenced page number, *p (*i.e., *p* is the index into the page table). This result, *b + p,* is the main memory address of the PTE for page *p. [Note:* For simplicity, we assume that the size of each entry in the page table is fixed and that each address in memory stores an entry of that size.] This PTE indicates that virtual page *p* corresponds to page frame *p'.* The system then concatenates *p'* with the displacement, *d,* to form the real address, *r.* This is an example of direct mapping.

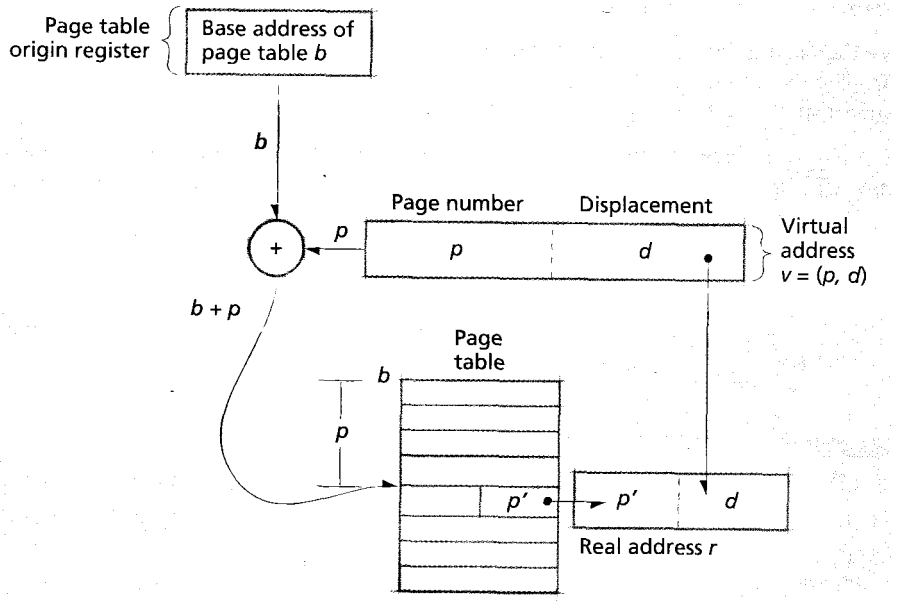


Figure 10.12 | Paging address translation by direct mapping.

because the page table contains one entry for every page in this process's virtual memory space, *V.* If the process contains *n* pages in *V,* then the direct-mapped page table for the process contains entries successively for page 0, page 1, page 2, ..., page *n — 1.* Direct mapping is much like accessing an array location via subscripting; the system can locate *directly* any entry in the table with a single access to the table.

The system maintains the virtual address being translated and the base address of the page table in high-speed registers on each processor, which enables operations on these values to be performed quickly (within a single instruction execution cycle). However, a system typically keeps the direct-mapped page table—which can be quite large—in main memory. Consequently, a reference to the page table requires one complete main memory cycle. Because the main memory access time ordinarily represents the largest part of an instruction execution cycle, and because we require an additional main memory access for page mapping, the use of direct-mapping page address translation can cause the computer system to run programs at about half speed (or even worse for machines that support multiple address instructions)! To achieve faster translation, one may implement the complete direct-mapped page table in high-speed cache memory. Due to the cost of nigh-speed cache memory and the potentially large size of virtual address spaces, maintaining the entire page table in cache memory is typically not viable. We discuss a solution to this problem in Section 10.4.3, Paging Address Translation with Direct/Associative Mapping.

Self Review

1. Why should the size of page table entries be fixed?
2. What type of special-purpose hardware is required for page address translation by direct mapping?

Ans: **1)** The key to virtual memory is that address translation must occur quickly. If the size of page table entries is fixed, the calculation that locates the entries is simple, which facilitates fast page address translation. **2)** A high-speed processor register is needed to store the base address of the page table.

10.4.2 Paging Address Translation by Associative Mapping

One way to increase the performance of dynamic address translation is to place the entire page table into a content-addressed (rather than location-addressed) **associative memory,** which has a cycle time greater than an order of magnitude faster than main memory.[28, 29, 30, 31] Figure 10.13 illustrates how dynamic address translation proceeds with pure **associative mapping.** A process refers to virtual address *v = (p,d).* Every entry in the associative memory is searched simultaneously for page *p.* The search returns *p′* as the page frame corresponding to page *p,* and *p′* is concatenated with *d,* forming the real address, *r.* Note that the arrows into the associative map enter every cell of the map, indicating that every cell of the associative memory is searched simultaneously for a match on *p.* This is what makes associative memory prohibitively expensive, even compared to direct-mapped cache. Because

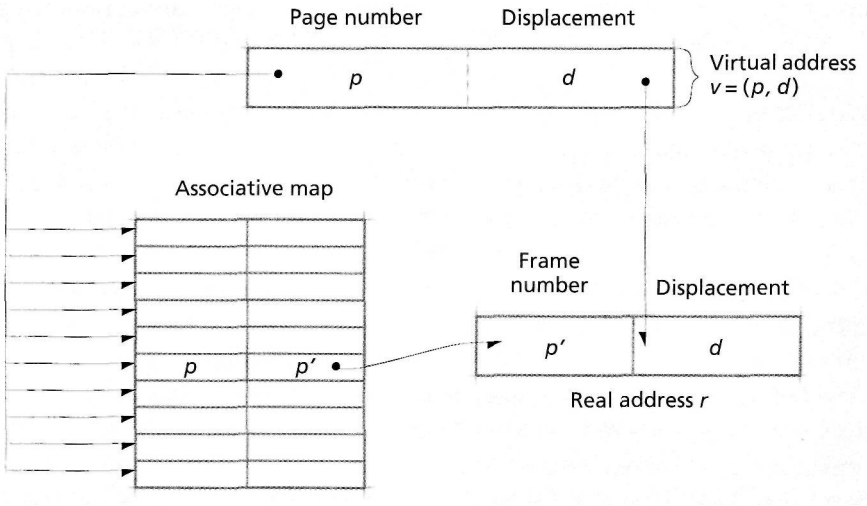


Figure 10.11 | *Paging address translation with pure associative mapping.*

of this expense, pure associative mapping is not used; we show it here in case its economics eventually becomes more favorable.

In the vast majority of systems, using a cache memory to implement pure direct mapping or an associative memory to implement pure associative mapping is too costly. As a result, many designers have chosen a compromise scheme that offers many of the advantages of the cache or associative memory approach at a more modest cost. We consider this scheme in the next section.

Self Review

1. Why is page address translation by pure associative mapping not used?
2. Does page address translation by pure associative mapping require any special-purpose hardware?

Ans: **1)** Associative memory is even more expensive than direct-mapped cache memory. Therefore, it would be prohibitively expensive to build a system that contained enough associative memory to store all of a process's PTEs. **2)** This technique requires an associative memory; however, it does not require a page table origin register to store the location of the start of the page table, because associative memory is content addressed, rather than location addressed.

10.4.3 Paging Address Translation with Direct/Associative Mapping

Much of the discussion to this point has dealt with the computer hardware required to implement virtual memory efficiently. The hardware view presented has been a logical rather a than physical one. We are concerned not with the precise structure of the devices but with their functional organization and relative speeds. This is the view the operating systems designer must have, especially as hardware designs evolve.

Historically, hardware has improved at a much more dramatic pace than improvements in software. Designers have become reluctant to commit themselves to a particular hardware technology because they expect that a better one will soon be available. Operating systems designers, however, must use the capabilities and economics of today's hardware. High-speed cache and associative memories simply are far too expensive to hold the complete address mapping data for full virtual address spaces. This leads to a compromise page-mapping mechanism.

the compromise uses an associative memory, called the **translation lookaside buffer (TLB),** capable of holding only a small percentage of the complete page table for a process (Fig. lO.f 4). The TLB's contents may be controlled by the oper-

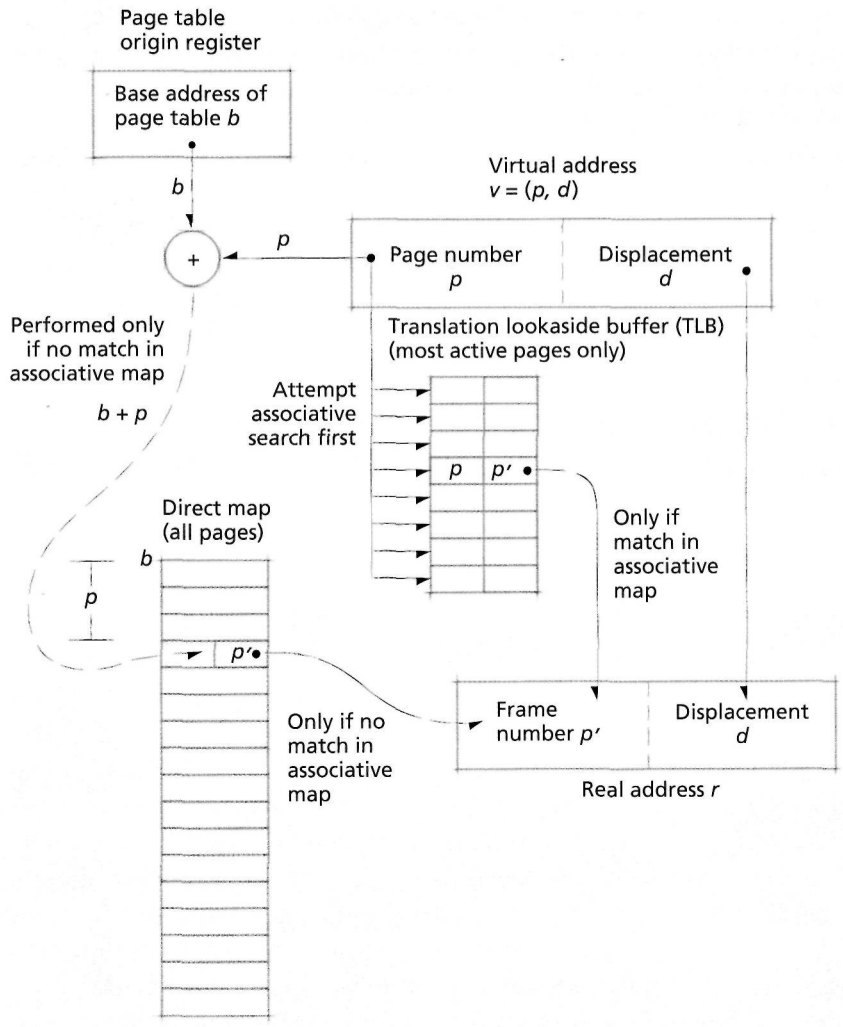


Figure 10.14 | Paging address translation with combined associative/direct mapping.

ating system or by hardware, depending on the architecture.[32] In this chapter, we assume that the TLB is managed by hardware. The page table entries maintained in this map typically correspond to the more-recently referenced pages only, using the heuristic that a page referenced recently is likely to be referenced again in the near future. This is an example of **locality** (more specifically of temporal locality—locality in time), a phenomenon discussed in detail in Chapter 11, Virtual Memory Management (see the Operating Systems Thinking feature, Empirical Results. Locality-Based Heuristics). The TLB is an integral part of today's MMUs.

Dynamic address translation under this scheme proceeds as follows. A process references virtual address $v = (p, d)$. The page mapping mechanism first tries to find page p in the TLB. If the TLB contains p, then the search returns p' as the frame number corresponding to virtual page p, and p' is concatenated with the displacement d to form the real address, r. When the system locates the mapping for p in the TLB, it experiences a **TLB hit.** Because the TLB stores entries in high-speed associative memory, this enables the translation to be performed at the fastest possible speed for the system. The problem, of course, is that because of prohibitive cost, the associative map can hold only a small portion of most virtual address spaces. The challenge is to pick a size within design's economic parameters that holds enough entries so that a large percentage of references will result in TLB hits.

If the TLB does not contain an entry for page p (i.e., the system experiences a **TLB miss),** the system locates the page table entry using a conventional direct map in slower main memory, which increases execution time. The address in the page

Operating Systems Thinking

Empirical Results: Locality-Based Heuristics

Some fields of computer science have significant bodies of theory, built from a mathematical framework. There are indeed aspects of operating systems to which such theoretical treatments apply. But for the most part, operating systems design is based on empirical, (i.e., observed) results and the heuristics that designers and implementers have developed the first operating systems were deployed in the 1950s. For example, we will study the empirical phenomena of spatial locality and temporal locality (i.e., locality in time), which are observed phenomena. If it is sunny in your town, it is likely—but not guaranteed—to be sunny in nearby towns. Also, if it is sunny in your town now, it is likely—but not guaranteed—that is was sunny in your town a short while ago and it will be sunny in your town a short while into the future. We will see many examples of locality in computer systems and we will study many locality-based heuristics in various areas of operating systems such as virtual memory management. Perhaps the most famous locality-based heuristic in operating systems is Peter Denning's working set theory of program behavior, which we discuss in Chapter 11.

table origin register, *b,* is added to the page number, *p,* to locate the appropriate entry for page *p* in the direct-mapped page table in main memory. The entry contains *p',* the page frame corresponding to virtual page *p**p'* is concatenated with the displacement, *d,* to form the real address, *r.* The system then places the PTE in the TLB so that references to page/? in the near future can be translated quickly. If the TLB is full, the system replaces an entry (typically the one least-recently referenced) to make room for the entry for current page.

Empirically, due to the phenomenon of locality, the number of entries in the TLB does not need to be large to achieve good performance. In fact, systems using this technique with only 64 or 128 TLB entries (which, assuming a 4KB page size, covers 256-512KB of virtual memory space) often achieve 90 percent or more of the performance possible with a complete associative map. Note that on a TLB miss, the processor must access main memory to obtain the page frame number. Depending on the hardware and memory management strategy, such misses can cost tens or hundreds of processor cycles, because main memory typically operates at slower speeds than processors do.[33-34-35]

Using a combined associative/direct mapping mechanism is an engineering decision based on the relative economics and capabilities of existing hardware technologies. Therefore, it is important for operating systems students and developers to be aware of such technologies as they emerge. The Recommended Reading and Web Resources sections provide several resources documenting these technologies.

Self Review

1. (T/F) The majority of a process's PTEs must be stored in the TLB to achieve high performance.
2. Why does the system need to invalidate a PTE in the TLB if a page is moved to secondary storage?

Ans: **1)** False. Processes often achieve 90 percent or more of the performance possible when only a small portion of a process's PTEs are stored in the TLB. **2)** If the system does not invalidate the PTE, a reference to the nonresident page will cause the TLB to return a page frame that might contain invalid data or instructions (e.g., a page belonging to a different sprocess).

10.4.4 Multilevel Page Tables

One limitation of address translations using direct mapping is that all of the PTEs of a page table must be in the map and stored contiguously in sequential order by page number. Further, these tables can consume a significant amount of memory. For example, consider a 32-bit virtual address space using 4KB pages. In this system, the page size is 2^{12} bytes, leaving 2^{32-12}, or 2^{20}, page numbers. Thus, each virtual address space would require approximately one million entries, one for each page, for a total addressability of about four billion bytes. A 64-bit virtual address space using 4MB pages would require approximately four trillion entries! Holding such a large page table in memory (assuming that there was enough memory to do so) can severely limit the memory available for actual programs; it also does not

makes sense, because processes may need to access only small portions of their address spaces at any given time to run efficiently.

Multilevel (or **hierarchical) page tables** enable the system to store in discontiguous locations in main memory those portions of a process's page table that the process is actively using. The remaining portions of can be created the first time they are used and moved to secondary storage when they cease being actively used. Multilevel page tables are implemented by creating a hierarchy—each level containing a table that stores pointers to tables in the level below. The bottom-most level is comprised of tables containing the page-to-page-frame mappings.

For example, consider a system that uses two levels of page tables (Fig. 10.15). The virtual address is the ordered triplet $v = (p,t, d)$, where the ordered pair (p, t) is the page number and d is the displacement into that page. The system first adds the value of p to the address in main memory of the start of the page directory, a, stored in the page directory origin register. The entry at location $a+p$ contains the address of the start of the corresponding page table, b. The system adds t to b to locate the page table entry that stores a page frame number, p'. Finally, the system forms the real address by concatenating p' and the displacement, d.

In most systems, each table in the hierarchy is the size of one page, which enables the operating system to transfer page tables between main memory and

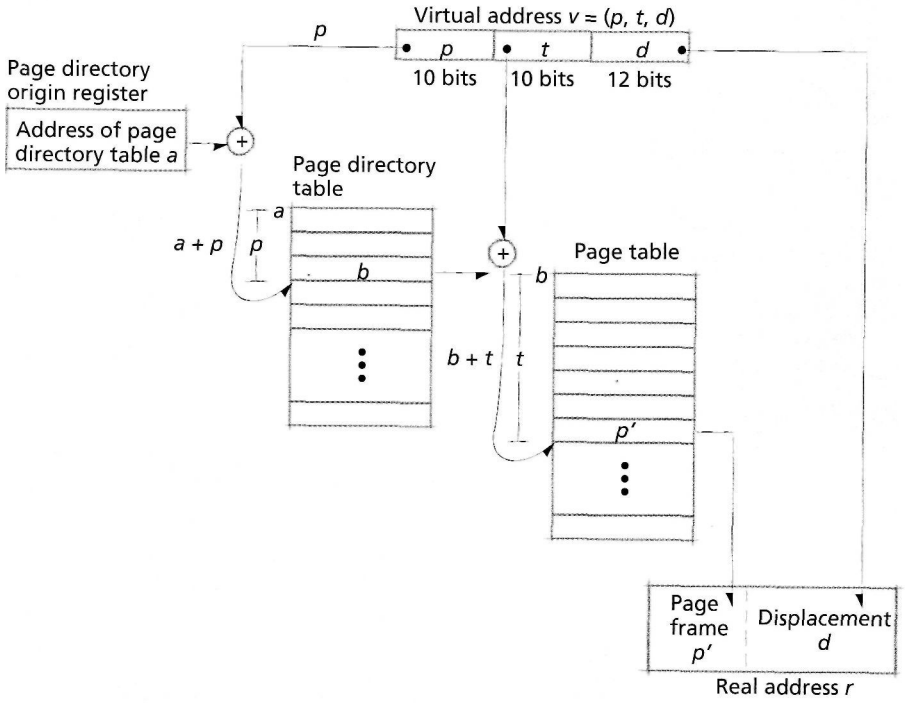


Figure 10.15 | *Multilevel page address translation.*

secondary storage easily. Let us examine how a two-level page table reduces memory consumption compared to direct mapping in a system that provides a 32-bit virtual address space using 4KB pages (which, again, requires 20-bit page numbers). Each page table would contain 2^{20}, or 1,048,576, entries for each process. Considering that typical systems contain tens, if not hundreds, of processes, and that high-end systems can contain thousands of processes, this could lead to considerable memory overhead. If each page table entry were 32 bits, this would consume 4MB of contiguous memory for each process's virtual address space. If the space were sparsely populated (i.e., few page table entries were in use), much of the page table would be unused, resulting in significant memory waste called **table fragmentation.**

Observe how table fragmentation is reduced using a two-level page table as follows. Each 32-bit virtual address is divided into a 10-bit offset into the page directory, *p,* a 10-bit offset into a page table, *t,* and a 12-bit displacement into the page, *d.* In this case, the page directory contains pointers to 2^{10} tables (one entry for each number specified by the 10-bit offset into the page directory, *p*), each of which holds 2^{10} PTEs (one entry for each number specified by the 10-bit offset into a page table, *t*). The first 10 bits of the virtual address is used as an index into the first table. The entry at this index is a pointer to the next table, which is a page table containing PTEs. The second 10 bits of the virtual address is used as an index into the page table that locates the PTE. Finally, the PTE provides the page frame number, *p′,* which is concatenated with the last 12 bits of the virtual address to determine the real address.

If a process uses no more than the first 1,024 pages in its virtual address space, the system need only maintain 1,024 (2^{10}) entries for the page directory and 1,024 page table entries, whereas a direct-mapped page table must maintain over one million entries. Multilevel page tables enable the system to reduce table fragmentation by over 99 percent.

The overhead incurred by multilevel page tables is the addition of another memory access to the page mapping mechanism. At first, it may appear that this additional memory cycle would result in worse performance than a direct-mapped page table. Due to locality of reference and the availability of a high-speed TLB, however, once a virtual page has been mapped to a page frame, future references to that page do not incur the memory access overhead. Thus, systems that employ multilevel page tables rely on an extremely low TLB miss rate to achieve high performance.

Multilevel page tables have become common in paged virtual memory systems. For example, the IA-32 architecture supports two levels of page tables (see Section 10.7, Case Study: IA-32 Intel Architecture Virtual Memory).[36]

Self Review

1. Discuss the benefits and drawbacks of using a multilevel paging system instead of a direct-mapped paging system.
2. A designer suggests reducing the memory overhead of direct-mapped page tables by increasing the size of pages. Evaluate the consequences of such a decision.

Ans: **1)** Multilevel paging systems require considerably less main memory space to hold mapping information than direct-mapped paging systems. However, multilevel paging systems require more memory accesses each time a process references a page whose mapping is not in the TLB, and potentially can run more slowly. **2)** Assuming that the size of each virtual address remains fixed, page address translation by direct mapping using large pages reduces the number of entries the system must store for each process. The solution also reduces memory access overhead compared to a multilevel page table. However, as page sizes increase, so do the possibility and magnitude of internal fragmentation. It is possible that the amount of wasted memory due to internal fragmentation could be equal to or greater than the tabic fragmentation due to storing entries for smaller pages.

10.4.5 Inverted Page Tables

As we discussed in the preceding section, multilevel page tables reduce the number of page table entries that must reside in main memory at once for each process when compared to direct-mapped page tables. In this case, we assumed that processes use only a small, contiguous region of their virtual address spaces, meaning that the system can reduce memory overhead by storing page table entries only for the region of a process's virtual address space that is in use. However, processes in scientific and commercial environments that modify large quantities of data might use a significant portion of their 32-bit virtual address spaces. In this case, multilevel page tables do not necessarily decrease table fragmentation. In 64-bit systems that contain several levels of page tables, the amount of memory consumed by mapping information can become substantial.

An **inverted page table** solves this problem by storing exactly one PTE for each page frame in the system. Consequently, the number of PTEs that must be stored in main memory is proportional to the size of physical memory, not to the size of a virtual address space. The page tables are inverted relative to traditional page tables because the PTEs are indexed by page frame number rather than virtual page number. Note that inverted page tables do not store the secondary storage location of nonresident pages. This information must be maintained by the operating system, and need not be in tables. For example, an operating system might use a binary tree to store the locations of nonresident pages so that they may be found quickly.

Inverted page tables use hash functions to map virtual pages to PTEs.[37,38] A **hash function** is a mathematical function that takes a number as an input and outputs a number, called a **hash value,** within a finite range. A **hash table** stores each item in the cell corresponding to the item's hash value. Because the domain of a hash function (e.g., a process's virtual page numbers) is generally larger than its range (e.g., the page frame numbers), multiple inputs can result in the same hash value—these are called **collisions.** To prevent multiple items from overwriting each other when mapped to the same cell of the hash table, inverted page tables can implement a variant of a **chaining** mechanism to resolve collisions as follows. When a hash value maps an item to a location that is occupied, a new function is applied to the hash value. The resulting value is used as the position in the table where the input is to be placed. To ensure that the item can be found when referenced, a pointer to this position is appended to the entry in the cell corresponding to the

item's original hash value. This process is repeated each time a collision occurs. Inverted page tables typically use linked lists to chain items.

In a paging system implementing inverted page tables, each virtual address is represented by the ordered pair v = *(p,d)*. To quickly locate the hash table entry corresponding to virtual page *p,* the system applies a hash function to *p,* which produces a value *q* (Fig. 10.16). If the *q*th cell in the inverted page table contains *p,* then the requested virtual address is in page frame *q*. If page number in the *qth* cell of the inverted page table does not match *p,* the system checks the value of the chaining pointer for that cell. If the pointer is null, then the page is not in memory, so the processor issues a page fault. The operating system can then retrieve the page from secondary storage. Otherwise, there has been a collision at that index in the inverted page table, so the page table entry stores a pointer to the next entry in the chain. The system follows the pointers in the chain until it finds an entry containing *p* or until the chain ends, at which point the processor issues a page fault to indicate that the page is not resident. The operating system can then retrieve the page from secondary storage. In Fig. 10.16, the system locates an entry corresponding to *p* after following one chaining pointer to cell *p′,* so *p* is located in page frame *p′*.

Although a careful choice of hash functions can reduce the number of collisions in the hash table, each additional chaining pointer requires the system to

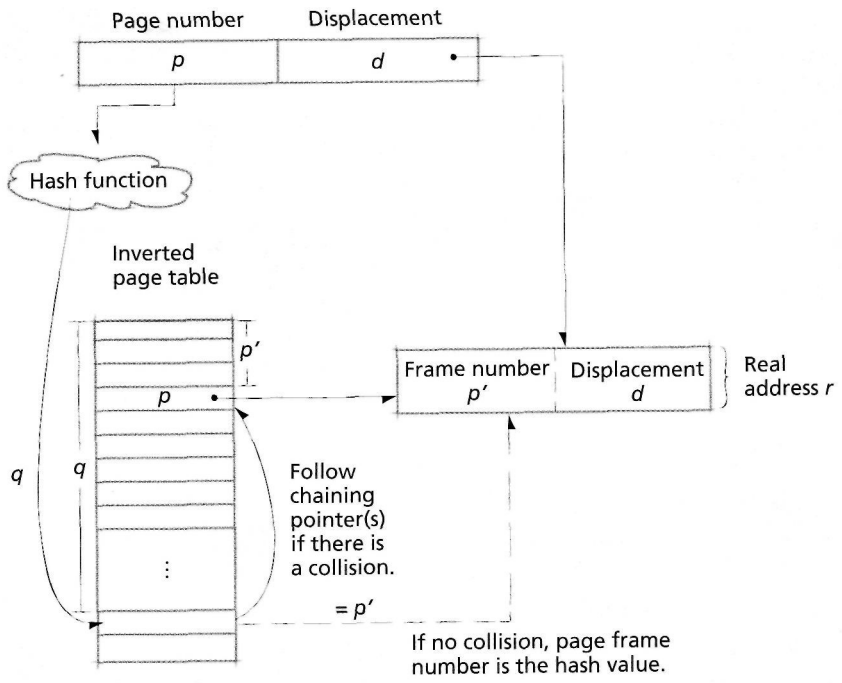


Figure 10.16 | *Page address translation using inverted page tables.*

access main memory, which can substantially increase address translation time. To improve performance, the system can attempt to reduce the number of collisions by increasing the range of hash values produced by the function. Because the size of the inverted page table must remain fixed to provide direct mappings to page frames, most systems increase the range of the hash function using a **hash anchor table,** which is a hash table containing pointers to entries in the inverted page table (Fig. 10.17). The hash anchor table imposes an additional memory access to virtual-to-physical address translation using inverted page tables and increases table fragmentation. If the hash anchor table is sufficiently large, however, the number of collisions in the inverted page table can be significantly reduced, which can speed address translation. The size of the hash anchor table must be chosen carefully to balance address translation performance with memory overhead.[39]-[40]

Inverted page tables are typically found in high-performance architectures, such as the Intel IA-64 and the HP PA-RISC architectures, but are also implemented in the PowerPC architecture, which is found in the consumer line of Apple computers.

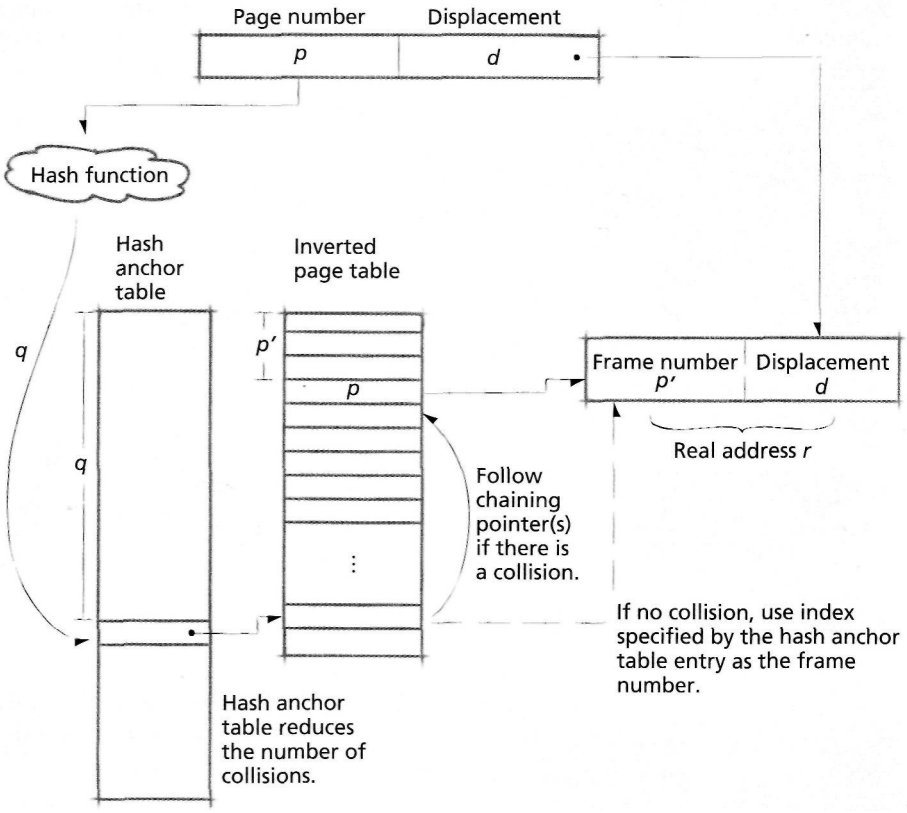


Figure 10.17 *| Inverted page table using a hash anchor table.*

Self Review

1. Compare and contrast inverted page tables (without a hash anchor table) to direct-mapped page tables in terms of memory efficiency and address translation efficiency.
2. Why are PTEs larger in inverted page tables than in direct-mapped page tables?

Ans: **1)** Inverted page tables incur less memory overhead than direct-mapped page tables because an inverted page table contains only one PTE for each physical page frame, whereas a direct-mapped page table contains one PTE for each virtual page. However, address translation may be much slower using an inverted page table than when using a direct-mapped table because the system may have to access memory several times to follow a collision chain. **2)** A PTE in an inverted page table must store a virtual page number and a pointer to the next PTE in the collision chain. A PTE in a direct-mapped page table need only store a page frame number and a resident bit.

10.4.6 Sharing in a Paging System

In multiprogrammed computer systems, especially in timesharing systems, it is common for many users to execute the same programs concurrently. If the system allocated individual copies of these programs to each user, much of main memory would be wasted. The obvious solution is for the system to share those pages common to individual processes.

The system must carefully control sharing to prevent one process from modifying data that another process is accessing. In most of today's systems that implement sharing, programs are divided into separate procedure and data areas. Nonmodifiable procedures are called pure procedures or reentrant procedures. Modifiable data cannot be shared without careful concurrency control. Nonmodifiable data can be shared. Modifiable procedures cannot be shared (although one might envision an esoteric example where doing this with some form of concurrency control would make sense).

All this discussion points to the need to identify each page as either sharable or nonsharable. Once each process's pages have been categorized in this fashion, then sharing in pure paging systems can be implemented as in Fig. 10.18. If the page table entries of different processes point to the same page frame, then this page frame is shared by each of the processes. Sharing reduces the amount of main memory required for a group of processes to run efficiently and can make it possible for a given system to increase its degree of multiprogramming.

One example in which page sharing can substantially reduce memory consumption is with the UNIX fork system call. When a process forks, the data and instructions for both the parent process and its child are initially identical. Instead of allocating an identical copy of data in memory for the child process, the operating system can simply allow the child process to share its parent's virtual address space while providing the illusion that each process has its own, independent virtual address space. This improves performance because it reduces the time required to initialize the child process and reduces memory consumption between the two processes (see the Operating Systems Thinking feature, Lazy Allocation). However, because the

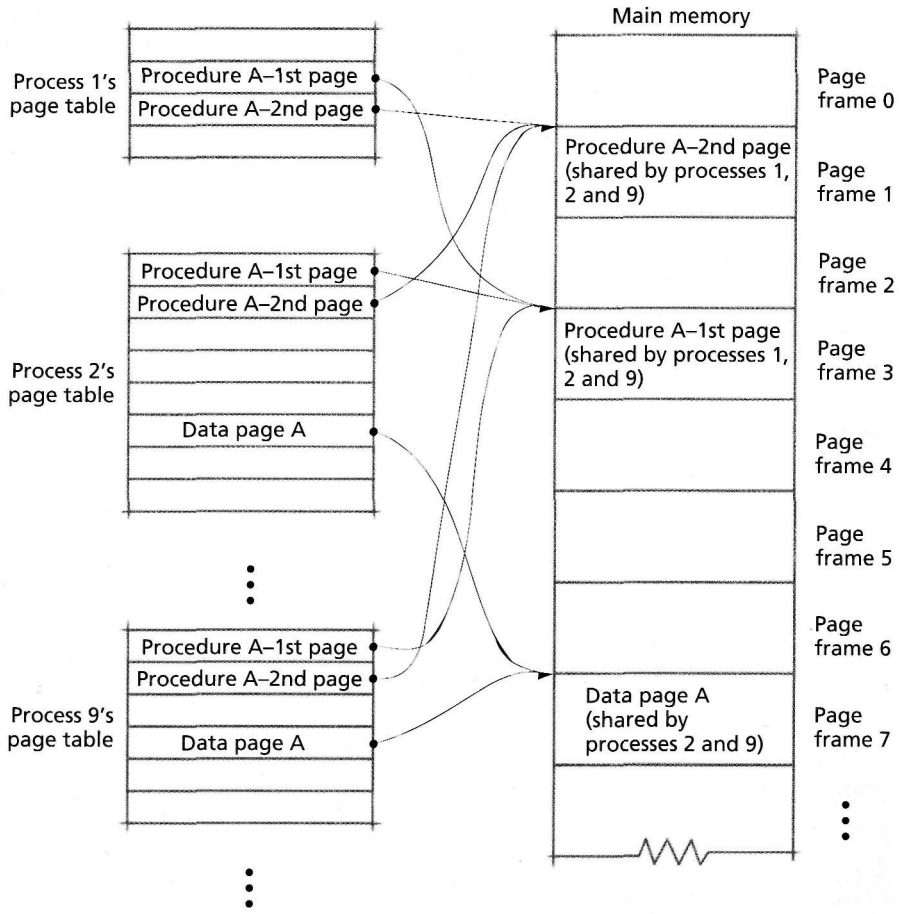


Figure 10.18 | Sharing in a pure paging system.

parent and child are unaware of the sharing, the operating system must ensure that the two processes do not interfere with each other when modifying pages.

Many operating systems use a technique called **copy-on-write** to address this problem. Initially, the system maintains one copy of each shared page in memory, as described previously, for processes P_1 through P_n. If P_1 attempts to modify a page of shared memory, the operating system creates a copy of the page, applies the modification and assigns the new copy to process P_1's virtual address space. The unmodified copy of the page remains mapped to the address space of all other processes sharing the page. This ensures that when one process modifies a shared page, no other processes are affected.

Many architectures provide each PTE with a **read/write bit** that the processor checks each time a process references an address. When the read/write bit is off, the page can be read, but not modified. When the bit is on, the page can be both read

and modified. Copy-on-write can thus be implemented by marking each shared page as read-only. When a process attempts to write to a page, the processor on which it executes will cause a page fault, which invokes the kernel. At this point, the kernel can determine that the process is attempting to write to a shared page and perform the copy-on-write.

Copy-on-write speeds process creation and reduces memory consumption for processes that do not modify a significant amount of data during execution. However, copy-on-write can result in poor performance if a significant portion of a process's shared data is modified during program execution. In this case, the process will suffer a page fault each time it modifies a page that is still shared. The overhead associated with invoking the kernel for each exception can quickly outweigh the benefits of copy-on-write. In the next chapter, we consider how sharing affects virtual memory management.

Self Review

1. Why might it be difficult to implement page sharing when using inverted page tables?
2. How does page sharing affect performance when the operating system moves a page from main memory to secondary storage?

Operating Systems Thinking

Lazy Allocation

Lazy resource allocation schemes allocate resources only at the last moment, when they are explicitly requested by processes or threads. For resources that ultimately will not be needed, lazy allocation schemes are more efficient than anticipatory resource allocation schemes, which might attempt to allocate those resources that would not be used, thus wasting the resources. For resources that ultimately will be needed, anticipatory schemes help processes and threads run more efficiently by having the resources they need available when they need them. so those processes and threads do not have to wait for the resources to be allocated. We discuss lazy allocation in various contexts in the book. One classic example of lazy allocation is demand paging, which we study in this chapter and the chapter that follows. With pure demand paging, a process's or thread's pages are brought to memory only as each is explicitly referenced. The advantage of demand paging is that the system never incurs the overhead of transferring a page from secondary storage to main memory unless that page will truly be needed. On the other hand, because such a scheme waits until a page is referenced before bringing it to main memory, the process will experience a significant execution delay as it waits for the disk I/O operation to complete. And while the process or thread is waiting, the portion of that process or thread memory that is in memory is tying up space that could otherwise be used by other running processes. Copy-on-write is another example of lazy allocation.

Ans: **1)** The type of page sharing discussed in this section is accomplished by having PTEs from different process's page tables point to the same page frame number. Because inverted page tables maintain exactly one PTE in memory for each page frame, the operating system must maintain sharing information in a data structure outside the inverted page table. One challenge of this type of page sharing is determining if a shared page is already in memory, because it was recently referenced by a different process, when another process first references it. **2)** When the operating system moves a shared page to secondary storage, it must update the corresponding PTE for every process sharing that page. If numerous processes share the page, this could incur significant overhead compared to that for an unshared page.

10.5 Segmentation

In the preceding chapter, we discussed how a variable-partition multiprogramming system can place a program in memory on a first-fit, best-fit or worst-fit basis. Under variable-partition multiprogramming, each program's memory and data occupy one contiguous section of memory called a partition. An alternative is physical memory **segmentation** (Fig. 10.19). Under this scheme, a program's data and instructions are divided into blocks called **segments.** Each segment consists of contiguous locations; however, the segments need not be the same size nor must they be placed adjacent to one another in main memory.

One advantage of segmentation over paging is that it is a logical rather than a physical concept. In their most general form, segments are not arbitrarily constrained to a certain size. Instead, they are allowed to be (within reasonable limits) as large or as small as they need to be. A segment corresponding to an array is as large as the array. A segment corresponding to a procedural code unit generated by a compiler is as large as it needs to be to hold the code.

In a virtual memory segmentation system, we have the ability to maintain in main memory only those segments that a program requires to execute at a certain time; the remainder of the segments reside on secondary storage.[41] A process may execute while its current instructions and data are located in a segment that resides in main memory. If a process references memory in a segment that does not currently reside in main memory, the virtual memory system must retrieve that segment from secondary storage. Under pure segmentation, the system transfers a segment from secondary storage as a complete unit and places all the locations within that segment in contiguous locations in main memory. An incoming segment may be placed in any available area in main memory that is large enough to hold it. The placement strategies for segmentation are identical to those used in variable-partition multiprogramming.[42]

A virtual memory segmentation address is an ordered pair v = *(s, d),* where *s* is the segment number in virtual memory in which the referenced item resides, and *d* is the displacement within segment *s* at which the referenced item is located (Fig. 10.20).

Self Review

1. (T/F) Segmented virtual memory systems do not incur fragmentation.
2. How does segmentation differ from variable-partition multiprogramming?

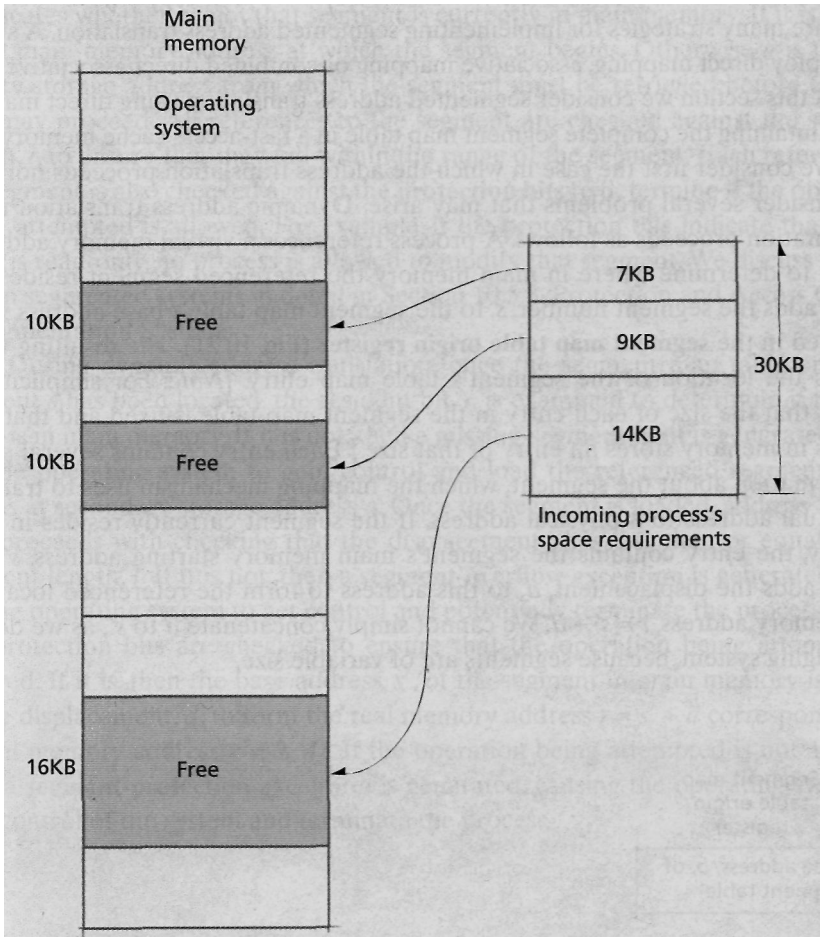


Figure 10.19 | Noncontiguous memory allocation in a real memory segmentation system.

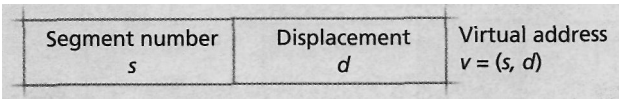


***Figure 10.20** | Virtual address format in a pure segmentation system.*

Ans: **1)** False. Segmented virtual memory systems can incur external fragmentation, exactly as in variable-partition multiprogramming systems. **2)** Unlike programs in a variable-partition multiprogramming system, programs in a segmented virtual memory system can be larger than main memory and need only a portion of their data and instructions in memory to execute. Also, in variable-partition multiprogramming systems, a process occupies one contiguous range of main memory, whereas in segmentation systems this need not ne so.

10.5.1 Segmentation Address Translation by Direct Mapping

There are many strategies for implementing segmented address translation. A system can employ direct mapping, associative mapping or combined direct/associative mapping. In this section we consider segmented address translation using direct mapping and maintaining the complete segment map table in a fast-access cache memory.

We consider first the case in which the address translation proceeds normally and consider several problems that may arise. Dynamic address translation under segmentation proceeds as follows. A process references a virtual memory address $v = (s, d)$ to determine where in main memory the referenced segment resides. The system adds the segment number, s, to the segment map table's base address value, b, located in the **segment map table origin register** (Fig. 10.21). The resulting value, $b + s$, is the location of the segment's table map entry. *[Note:* For simplicity, we assume that the size of each entry in the segment map table is fixed and that each address in memory stores an entry of that size.] Each entry contains several pieces of information about the segment, which the mapping mechanism uses to translate the virtual address to a physical address. If the segment currently resides in main memory, the entry contains the segment's main memory starting address, s'. The system adds the displacement, d, to this address to form the referenced location's real memory address, $r = s' + d$. We cannot simply concatenate d to s', as we do in a pure paging system, because segments are of variable size.

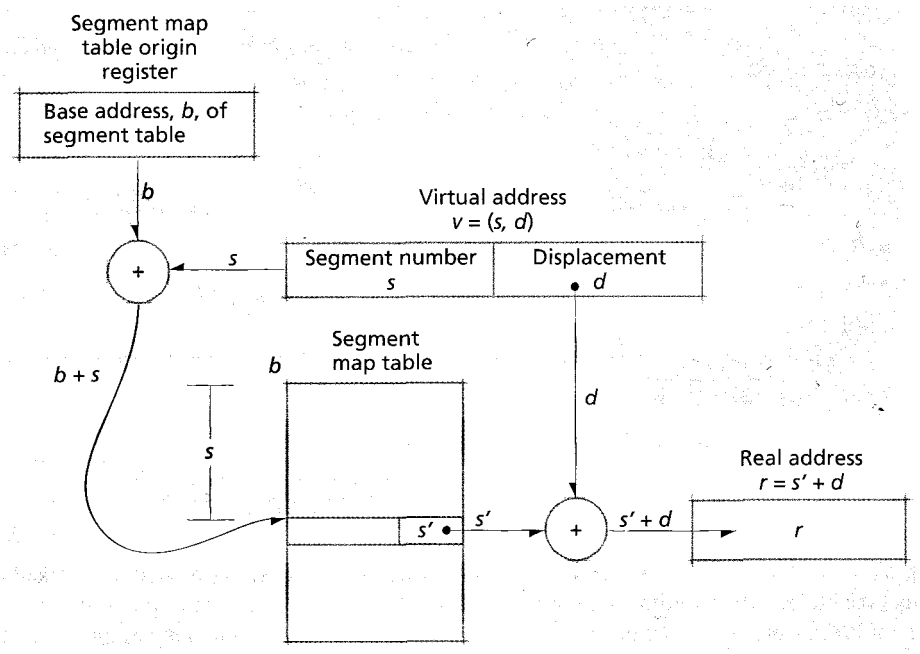


Figure 10.21 | *Virtual address translation in a pure segmentation system.*

Figure 10.22 shows a typical segment map table entry in detail. A resident bit, *r*, indicates whether or not that segment is currently in main memory. If it is, then *s′* is the main memory address at which the segment begins. Otherwise *a* is the secondary storage address from which the segment must be retrieved before the process may proceed. All references to the segment are checked against the segment length, *l*, to ensure that they fall within the range of the segment. Each reference to the segment is also checked against the **protection bits** to determine if the operation being attempted is allowed. For example, if the protection bits indicate that a segment is read-only, no process is allowed to modify that segment. We discuss protection in segmented systems in detail in Section 10.5.3, Protection and Access Control in Segmentation Systems.

During dynamic address translation, once the segment map table entry for segment *s* has been located, the resident bit, *r,* is examined to determine if the segment is in main memory. If it is not, then a **missing-segment fault** is generated, causing the operating system to gain control and load the referenced segment which begins at secondary storage address *a.* Once the segment is loaded, address translation proceeds with checking that the displacement, *d,* is less than or equal to the segment length, *l*. If it is not, then a **segment-overflow exception** is generated, causing the operating system to get control and potentially terminate the process. If it is, the protection bits are checked to ensure that the operation being attempted is allowed. If it is, then the base address, *s′,* of the segment in main memory is added to the displacement, *d,* to form the real memory address $r = s' + d$ corresponding to virtual memory address $v = (s, d)$. If the operation being attempted is not allowed, then a **segment-protection exception** is generated, causing the operating system to gain control of the system and terminate the process.

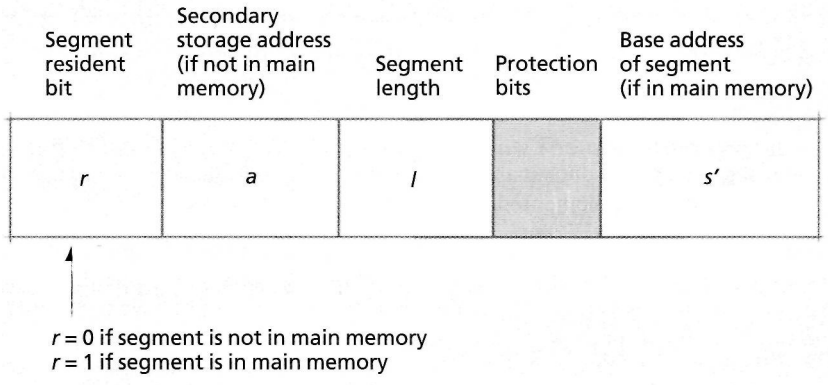


Figure 10.22 | *Segment map table entry.*

SelReview

1. Why is it incorrect to form the real address, *r,* by concatenating *s′* and *d*?
2. Does the page mapping mechanism require that *a* and *s′* be stored in separate cells of a segment map table entry, as shown in Fig. 10.22?

Ans: **1)** Unlike the page frame number *p′*, the segment base address *s′* is an address in main memory. Concatenating *s′* and *d* would result in an address beyond the limit of main memory. Again, concatenation works with paging because the page size is a power of two and the number of bits reserved for the displacement on a page and the number of bits reserved for the page frame number add up to the number of bits in the virtual address. **2)** No. To reduce the amount of memory consumed by segment map table entries, many systems use one cell to store either the segment base address or the secondary storage address. If the resident bit is on, the segment map table entry stores a segment base address, but not a secondary storage address. If the resident bit is off, the segment map table entry stores a secondary storage address, but not a segment base address.

10.5.2 Sharing in a Segmentation System

Sharing segments can incur less overhead than sharing in a direct-mapped pure paging system. For example, to share an array that is stored in three and one-half pages, a pure paging system must maintain separate sharing data for each page on which the array resides. The problem is compounded if the array is dynamic, because the sharing information must be adjusted at execution time to account for the growing or shrinking number of pages that the array occupies. In a segmentation system, on the other hand, data structures may grow and shrink without changing the sharing information associated with the structure's segment.

Figure 10.23 illustrates how a pure segmentation system accomplishes sharing. Two processes share a segment when their segment table entries point to the same segment in main memory.

Although sharing provides obvious benefits, it also introduces certain risks. For example, one process could intentionally or otherwise perform an operation on a segment that negatively affects other processes sharing that segment. Therefore, a system that provides sharing also should provide appropriate protection mechanisms, to ensure that only authorized users may access or modify a segment.

SelfReview

1. How does segmentation reduce sharing overhead compared to sharing under pure paging?
2. Can copy-on-write be implemented using segments, and, if so, how?

Ans: **1)** Segmentation enables an entire block of shared memory to fit inside one segment. so the operating system maintains sharing information for one segment. Under paging, this segment might consume several pages, so the operating system would have to maintain sharing information for each page. **2)** Yes. Copy-on-write can be implemented by allocating a copy of the parent's segment map table to its child. If process P_1 (which can be a parent or child process) attempts to modify segment *s,* the operating system must create a new copy of the segment, located at main memory address *s′.* The operating system then changes entry *s* in P_1's segment map table to contain the address *s′.*

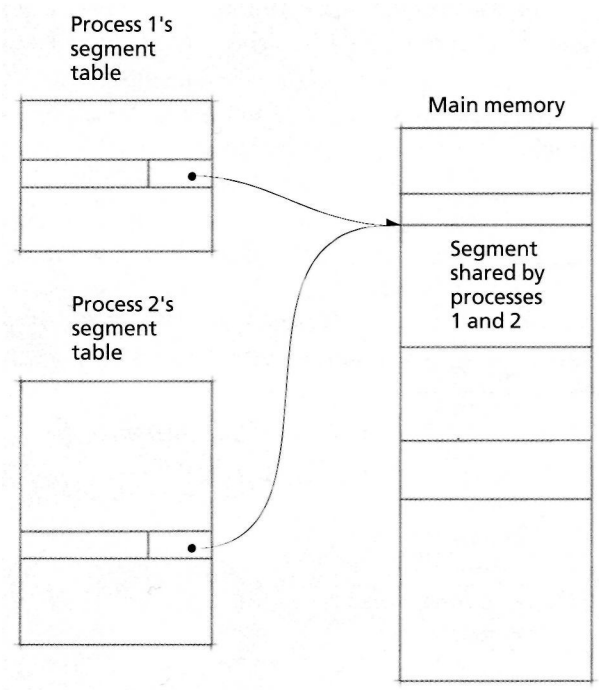


Figure 10.23 | *Sharing in a pure segmentation system.*

10.5.3 Protection and Access Control in Segmentation Systems

Segmentation promotes program modularity and enables better memory use through noncontiguous allocation and sharing. With these benefits, however, comes increased complexity. For example, a single pair of bounds registers no longer suffices to protect each process from destruction by other processes. Similarly, it becomes more difficult to limit the range of access of any given program. One scheme for implementing memory protection in segmentation systems is the use of memory **protection** keys (Fig. 10.24). In this case, each process is associated with a value, called a protection key. The operating system strictly controls this key, which can be manipulated only by privileged instructions. The operating system employs protection keys as follows. At context-switch time, the operating system loads the process's protection key into a processor register. When the process references a particular segment, the processor checks the protection key of the block containing the referenced item. If the protection key for the process and the requested block are the same, the process can access the segment. For example, in Fig. 10.24, Process 2 can access only those blocks with a protection-key value of 2. If the process attempts to access a block with a different protection key, the hardware prevents the memory access and vectors into the kernel (caused by a segment-protection exception). Although memory protection keys are not the most common protection

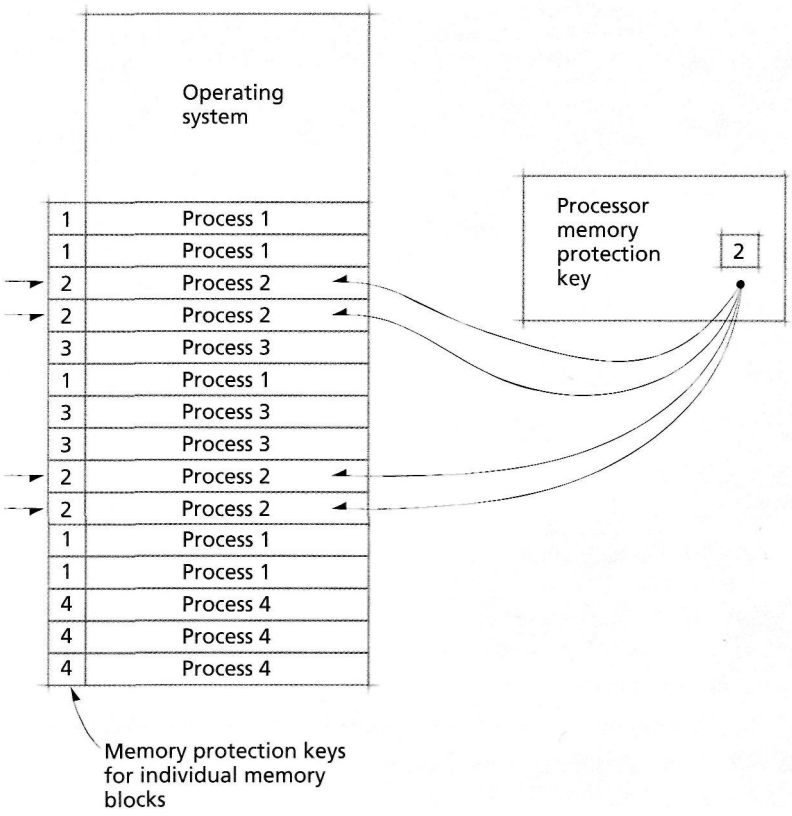


Figure 10.24 | *Memory protection with keys in noncontiguous memory allocation*

mechanism in today's systems, they are implemented in the IA-64 Intel architecture (i.e., the Itanium line of processors) and are generally intended for systems containing one virtual address space.[43]

The operating system can exercise further protection control by specifying how a segment may be accessed and by which processes. This is accomplished by assigning each process certain **access rights** to segments. Figure 10.25 lists the most common access control types in use in today's systems. If a process has **read access** to a segment, then it may read data from any address in that segment. If it has **write access** to a segment, then the process may modify any of the segment's contents and may add further information. A process given **execute access** to a segment may pass program control to instructions in that segment for execution on a processor. Execute access to a data segment is normally denied. A process given **append access** to a segment may write additional information to the end of the segment, but may not modify existing information.

By either allowing or denying each of these four access types, it is possible to create 16 different **access control modes.** Some of these are interesting, while others do not make sense. For simplicity, consider the eight different combinations of read, write, and execute access shown in Fig. 10.26.

In mode 0, no access is permitted to the segment. This is useful in security schemes in which the segment is not to be accessed by a particular process. In mode 1, a process is given **execute-only access** to the segment. This mode is useful when a process is allowed to execute instructions in the segment, but may not copy or modify it. Modes 2 and 3 are not useful—it does not make sense to give a process the right to modify a segment without also giving it the right to read the segment. Mode 4 allows a process **read-only access** to a segment. This is useful for accessing nonmodifiable data. Mode 5 allows a process **read/execute access** to a segment. This is useful for

Type of access	*Abbreviation*	*Description*
Read	**R**	This segment may be read.
Write	**W**	This segment may be modified.
Execute	**E**	This segment may be executed.
Append	**A**	This segment may have information added to its end.

Figure 10.25 | *Access control types.*

Mode	***Read***	*Write*	*Execute*	***Description***	*Application*
Mode 0	No	No	No	No access permitted	Security.
Mode 1	No	No	Yes	Execute only	A segment made available to processes that cannot modify it or copy it, but that can run it.
Mode 2	No	Yes	No	Write only	These possibilities are not useful, because granting write access without read access is impractical.
Mode 3	No	Yes	Yes	Write/execute but cannot be read	
Mode 4	Yes	No	No	Read only	Information retrieval.
Mode 5	Yes	No	Yes	Read/execute	A program can be copied or executed but cannot be modified.
Mode 6	Yes	Yes	No	Read/write but no execution	Protects data from an erroneous attempt to execute it.
Mode 7	Yes	Yes	Yes	Unrestricted access	This access is granted to trusted users.

Figure 10.26 | *Combining read, write and execute access to yiels useful access control modes.*

reentrant code. A process may make its own copy of the segment which it may then modify. Mode 6 allows a process **read/write access** to a segment. This is useful when the segment contains data that may be read or written by the process but that must be protected from accidental execution (because the segment does not contain instructions). Mode 7 allows a process **unrestricted access** to a segment. This is useful for allowing a process complete access to its own segments (such as self-modifying code) and for giving it most-trusted status to access other processes' segments.

The simple access control mechanism described in this section is the basis of segment protection implemented in many systems. Figure 10.27 shows one way a system can implement access control, namely by including protection bits in a segment's map table entry. The entry includes four bits—one for each type of access control. The system can now enforce protection during address translation. When a process makes a reference to a segment in virtual memory, the system checks the protection bits to determine if the process has proper authorization. For example, if a process is attempting to execute a segment that has not been granted execution rights, then it does not have authorization to perform its task. In this case, a segment-protection exception is generated, causing the operating system to gain control and terminate the process.

Self Review

1. Which access rights are appropriate for a process's stack segment?
2. What special-purpose hardware is required to implement memory protection keys?

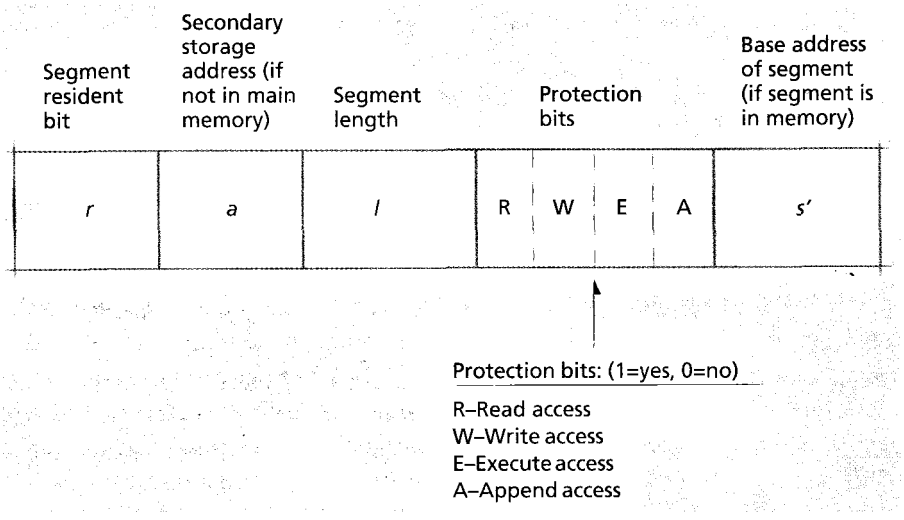


Figure 10.27 | *Segment map table entry with protection bits.*

Ans: **1)** A process should be able to read and write data in its stack segment and append new stack frames to the segment. **2)** A high-speed register is required to store the current process's memory protection key.

10.6 Segmentation/Paging Systems

Both segmentation and paging offer significant advantages as virtual memory organizations. Beginning with systems constructed in the mid-1960s, in particular Multics and IBM's TSS, many computer systems have been built that combine paging and segmentation.[44,45,46,47,48,49] These systems offer the advantages of the two virtual memory organization techniques we have presented in this chapter. In a combined segmentation/paging system, segments are usually arranged across multiple pages. All the pages of a segment need not be in main memory at once, and virtual memory pages that are contiguous in virtual memory need not be contiguous in main memory. Under this scheme, a virtual memory address is implemented as an ordered triple $v = (s,p,d)$, where s is the segment number, p the page number within the segment and d the displacement within the page at which the desired item is located (Fig. 10.28).

10.6.1 Dynamic Address Translation in a Segmentation/Paging System

Consider the dynamic address translation of virtual addresses to real addresses in a paged and segmented system using combined associative/direct mapping, as illustrated in Fig. 10.29.

A process references virtual address v = (s, p, d). The most recently referenced rages have entries in the associative memory map (i.e., the TLB). The translation mechanism performs an associative search to attempt to locate (s, p). If the TLB contains (s,p), then the search returns p', the page frame in which page p resides. This value is concatenated with the displacement, d, to form the real memory address r.

If the TLB does not contain an entry for (s,p), the processor must perform a complete direct mapping as follows. The base address, b, of the segment map table (in main memory) is added to the segment number, s, to form the address, $b+s$. This address corresponds to the physical memory location of the segment's entry in the segment map table. The segment map table entry indicates the base address, s', of the page table (in main memory) for segment s. The processor adds the page number, p, to the base address, s', to form the address of the page table entry for page p of segment s. This table entry indicates that p' is the page frame number corresponding to

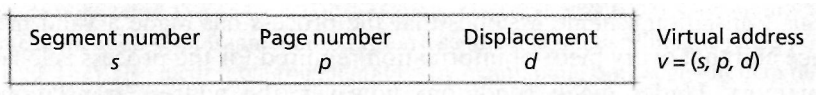


Figure 10.28 | *Virtual address format in a segmentation/paging system.*

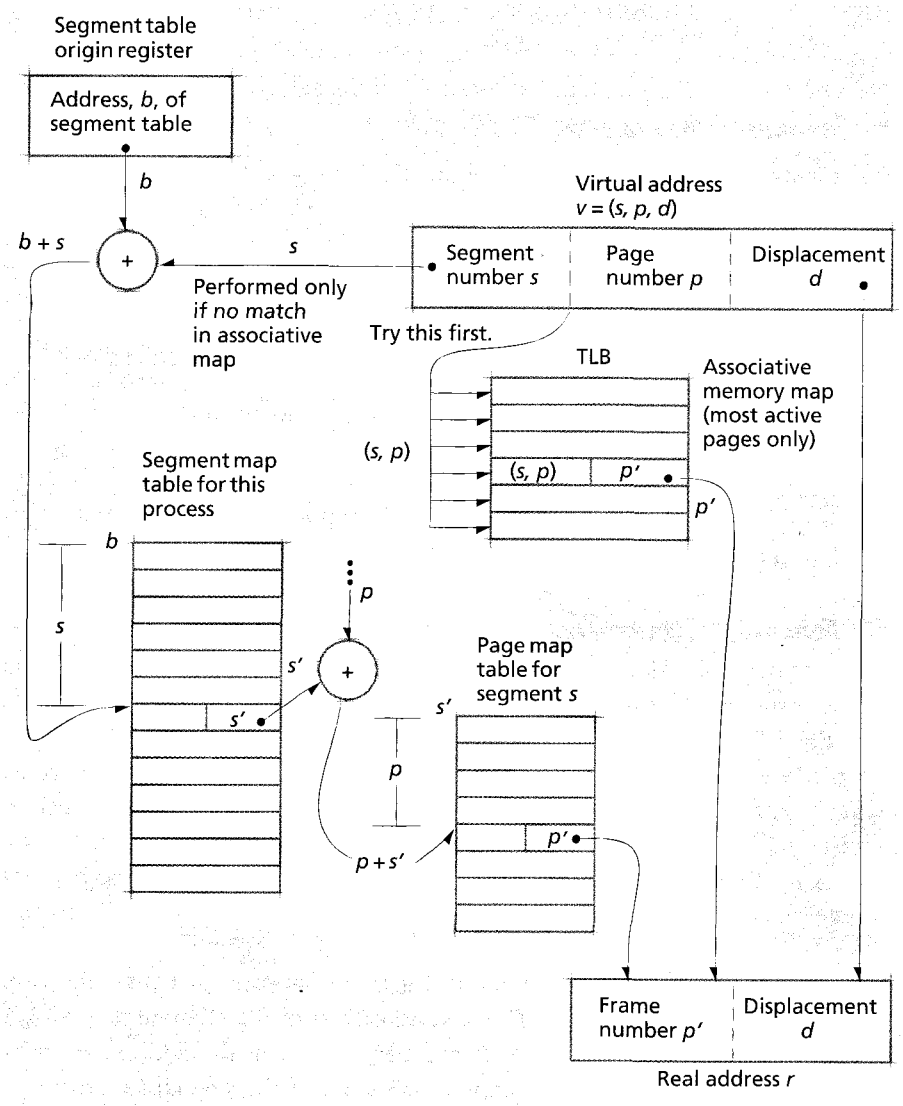


Figure 10.29 | *Virtual address translation with combined associative/direct mapping in a segmentation/paging system.*

virtual page *p*. This frame number, *p′*, is concatenated with the displacement, *d*, to form the real address, *r*. The translation is then loaded into the TLB.

This translation scheme assumes that the process has made a valid memory reference and that every piece of information required for the process is located in main memory. Under many conditions however, the address translation may require extra steps or will fail. The segment map table search may indicate that segment *s* is not in main memory, thus generating a missing-segment fault and causing

the operating system to locate the segment on secondary storage, create a page table for the segment, and load the appropriate page into main memory. If the segment is in main memory, then the reference to the page table may indicate that the desired page is not in main memory, initiating a page fault. This would cause the operating system to gain control, locate the page on secondary storage, and load it into main memory. It is also possible that a process has referenced a virtual memory address that extends beyond the range of the segment, thus generating a segment-overflow exception. Or the protection bits may indicate that the operation to be performed on the referenced virtual address is not allowed, thus generating a segment-protection exception. The operating system must handle all these possibilities.

The associative memory (or, similarly, a high-speed cache memory) is critical to the efficient operation of this dynamic address translation mechanism. If a purely direct mapping mechanism were used, with the complete map being maintained in main memory, the average virtual memory reference would require a memory cycle to access the segment map table, a second one to reference the page table and a third to reference the desired item in main memory. Thus every reference to an item would involve three memory cycles, and the computer system would run only at a small fraction of its normal speed; this would invalidate the benefits of virtual memory.

Figure 10.30 indicates the detailed table structure required by segmentation/paging systems. At the top level is a process table that contains an entry for every process in the system. Each process table entry contains a pointer to its process's segment map table. Each entry of a process's segment map table points to the page table for the associated segment, and each entry in a page table points either to the page frame in which that page resides or to the secondary storage address at which the page may be found. In a system with a large number of processes, segments and pages, this table structure can consume a significant portion of main memory. The benefit of maintaining all the tables in main memory is that address translation proceeds faster at execution time. However, the more tables a system maintains in main memory, the fewer processes it can support, and thus productivity declines. Operating systems designers must evaluate many such trade-offs to achieve the delicate balance needed for a system to run efficiently and to provide responsive service to system users.

Self Review

1. What special-purpose hardware is required for segmentation/paging systems?
2. In what ways do segmentation/paging systems incur fragmentation?

Ans: **1)** Segmentation/paging systems require a high-speed register to store the base address of the segment map table, a high-speed register to store the base address of the corresponding page table and an associative memory map (i.e., a TLB). **2)** Segmentation/paging systems can incur internal fragmentation when a segment is smaller than the page(s) in which it is placed. They also incur table fragmentation by maintaining both segment map tables and page tables in memory. Segmentation/paging systems do not incur external fragmentation (assuming the system uses one page size), because ultimately the memory is divided into fixed-size page frames, which can accommodate any page of any segment.

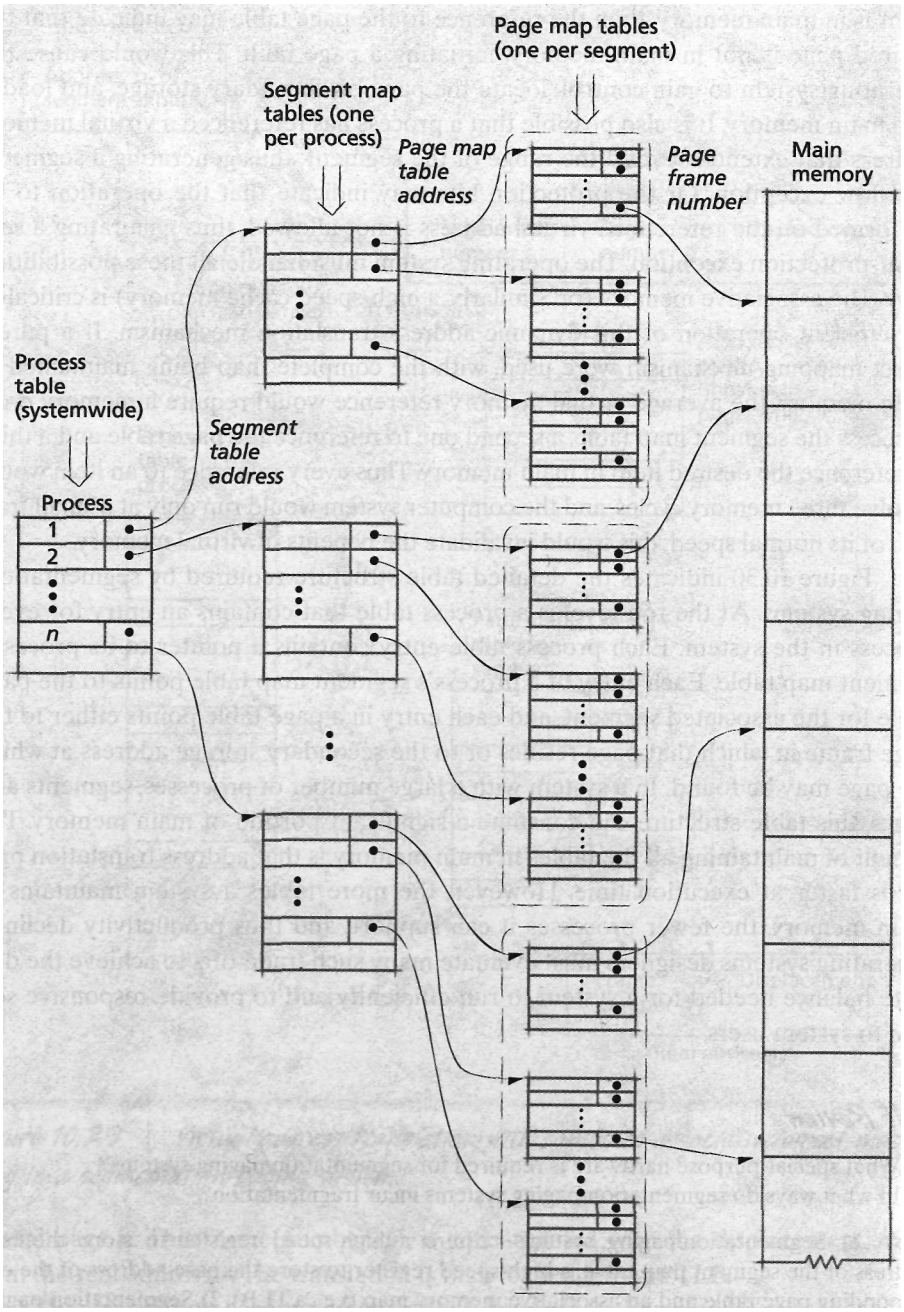


Figure 10.30 | *Table structure for a segmentation/paging system.*

10.6.2 Sharing and Protection in a Segmentation/Paging System

Segmentation/paging virtual memory systems take advantage of the architectural simplicity of paging and the access control capabilities of segmentation. In such a system the benefits of segment sharing become important. Two processes share memory when each process has a segment map table entry that points to the same page table, as is indicated in Fig. 10.31.

Sharing, whether in paged systems, segmented systems, or segmentation/paging systems, requires careful management by the operating system. In particular, consider what would happen if an incoming page were to replace a page shared by many processes. In this case, the operating system must update the resident bit in the corresponding page table entries for each process sharing the page. The operating system can incur substantial overhead determining which processes are sharing the page and which PTEs in their page tables must be changed. As we discuss in Section 20.6.4, Swapping, Linux reduces this overhead by maintaining a linked list of PTEs that map to a shared page.

Self Review

1. Why are segmentation/paging systems appealing?
2. What are the benefits and drawbacks of maintaining a linked list of PTEs that map to a shared page?

Ans: **1)** Segmentation/paging systems offer the architectural simplicity of paging and the access control capabilities of segmentation. **2)** A linked list of PTEs enables the system to update PTEs quickly when a shared page is replaced. Otherwise, the operating system would need to search each process's page table to determine if any PTEs need to be updated, which can incur substantial overhead. A drawback to the linked list of PTEs is that it incurs memory overhead. In today's systems, however, the cost of accessing main memory to search page tables generally outweighs the memory overhead incurred by maintaining the linked list.

10.7 Case Study: IA-32 Intel Architecture Virtual Memory

In this section, we discuss the virtual memory implementation of the IA-32 Intel architecture (i.e., the Intel Pentium line of processors), which supports either a pure segmentation or a segmentation/paging virtual memory implementation.[50] The set of addresses contained in each segment is called a **logical address space,** and its size depends on the size of the segment. Segments are placed in any available location in the system's **linear address space,** which is a 32-bit (i.e., 4GB) virtual address space. Under pure segmentation, the processor uses linear addresses to access main memory. If paging is enabled, the linear address space is divided into fixed-size page frames that are mapped to main memory.

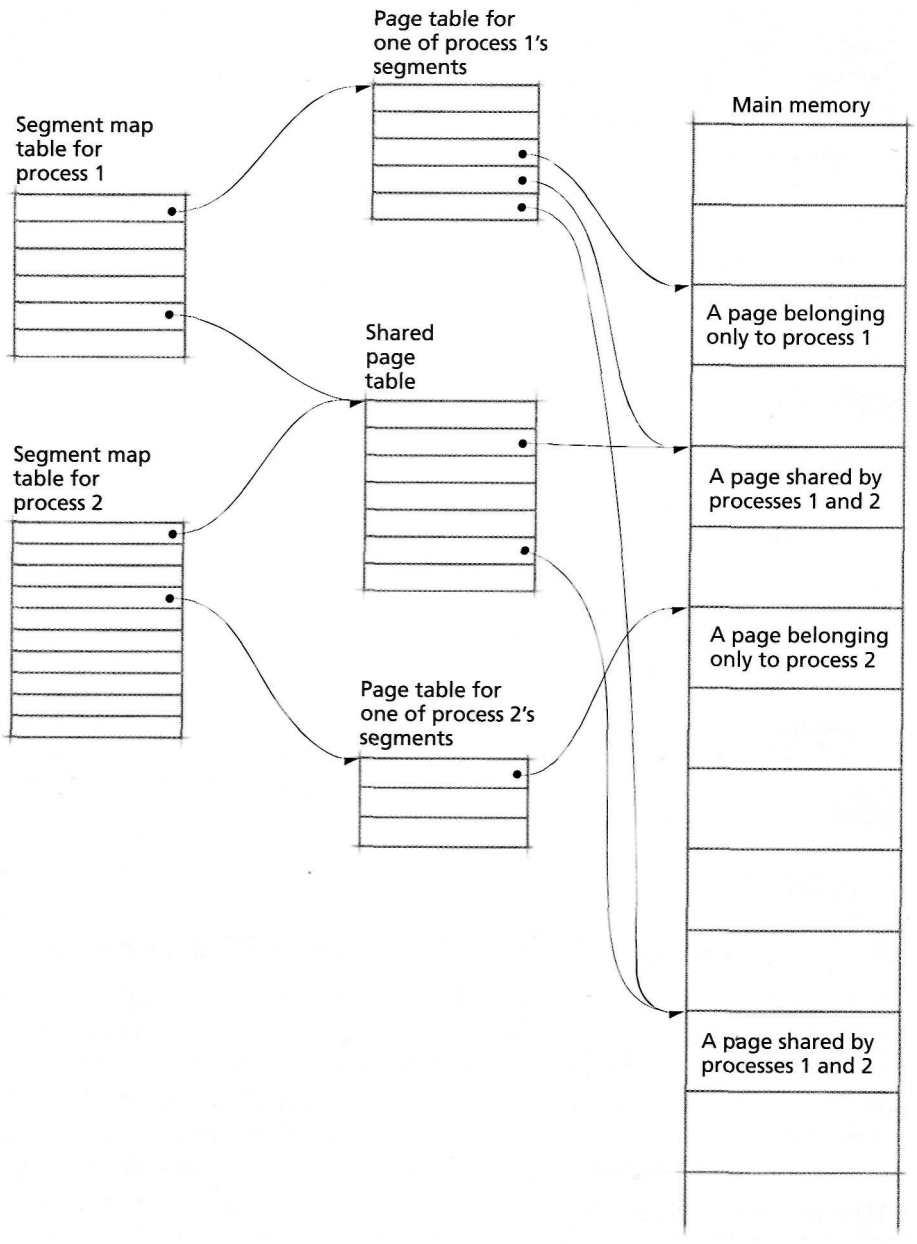


Figure 10.31 | *Two processes sharing a segment in a segmentation/paging system.*

The first stage of dynamic address translation maps segmented virtual addresses to the linear address space. Because there is no way to disable segmentation in the IA-32 specification, this translation occurs for every memory reference. Each segmented virtual address can be represented by the ordered pair v = *(s, d),* as discussed in Section 10.5.1. In the IA-32 architecture, *s* is specified by a 16-bit **segment selector** and *d* by a 32-bit offset (i.e., the displacement within segment *s'* at which the referenced item is located). The segment index, which is the 13 most-significant bits of the segment selector, specifies an 8-byte entry in the segment map table.

To speed address translation under segmentation, the IA-32 architecture provides six **segment registers,** named CS, DS, SS, ES, FS and GS, to store a process's segment selectors. The operating system normally uses the CS register to store a process's code segment selector (which typically corresponds to the segment containing its executable instructions), the DS register to store a process's data segment selector (which typically corresponds to the segment containing the process's data) and the SS register to store a process's stack segment selector (which typically corresponds to the segment containing its execution stack). The ES, FS and GS registers can be used for any other process segment selectors. Before a process can reference an address, the corresponding segment selector must be loaded into a segment register. This enables the process to reference addresses using the 32-bit offset, so that it does not have to specify the 16-bit segment selector for each memory reference.

To locate a segment map table entry, the processor multiplies the segment index by 8 (the number of bytes per segment map table entry) and adds that value to *b,* the address stored in the segment map table origin register. The IA-32 architecture provides two segment map table origin registers, the **global descriptor table register (GDTR)** and the **local descriptor table register (LDTR).** The values of the GDTR and LDTR are loaded at context-switch time.

The system's primary segment map table is the **global descriptor table (GDT)** which contains 8,192 (2^{13}) 8-byte entries. The first entry is not used by the processor—references to a segment corresponding to this entry will generate an exception. Operating systems load the value corresponding to this entry into unused segment registers (e.g., ES, FS and GS) to prevent processes .from accessing an invalid segment. If the system uses one segment map table for all of its processes, then the GDT is the only segment map table in the system. For operating systems that must maintain more than 8,192 segments, or that maintain a separate segment map table for each process, the IA-32 architecture provides **local descriptor tables (LDTs),** each containing 8,192 entries. In this case, the base address of each LDT must be stored in the GDT, because the processor places the LDT in a segment. To perform address translation quickly, the base address of the LDT, *b,* is placed in the LDTR. The operating system can create up to 8,191 separate segment map tables using LDTs (recall that the first entry in the GDT is not used by the processor).

Each segment map table entry, called a **segment descriptor,** stores a 32-bit **base address,** *s',* that specifies the location in the linear address space of the start of the segment. The system forms a linear address by adding the displacement, *d,* to the segment base address, *s'.* Under pure segmentation, the linear address, *s' + d,* is the real address.

The processor checks each reference against the segment descriptor's resident and protection bits before accessing main memory. The segment descriptor's **present bit** indicates whether the segment is in main memory; if it is, the address translation proceeds normally. If the present bit is off, the processor generates a **segment-not-present exception,** so that the operating system can load the segment from secondary storage. *[Note:* The terms "fault" and "exception" are used differently in the IA-32 specification than in the rest of the text (see Section 3.4.2, Interrupt Classes). The segment-not-present exception corresponds to a missing-segment fault.] The processor also checks each reference against the segment descriptor's 20-bit **segment limit,** *l*, which specifies the size of the segment. The system uses the segment descriptor's **granularity bit** to determine how to interpret the value specified in the segment's limit. If the granularity bit is off, segments can range in size from 1-byte ($l = 0$) to 1MB ($l = 2^{20} - 1$), in 1 byte increments. In this case, if $d > l$, the system generates a **general protection fault (GPF)** exception, indicating that a process has attempted to access memory outside of its segment. [*Note:* Again, the term "fault" is used differently in the IA-32 specification than in the rest of the text.] If the granularity bit is on, segments can range in size from 4KB ($l = 0$) to 4GB ($l = 2^{20} - 1$), in 4KB increments. In this case, if $d > (l \times 2$), the processor generates a GPF exception, indicating that a process has attempted to access memory outside of its segment.

The IA-32 architecture maintains a process's access rights to a segment in the **type** field of each segment map table entry. The type field's **code/data bit** determines whether the segment contains executable instructions or program data. Execute access is enabled when this bit is on. The type field also contains a **read/write bit** that determines whether the segment is read-only or read/write.

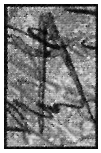
If paging is enabled, the operating system can divide the linear address space (i.e., the virtual address space in which segments are placed) into fixed-size pages. As we discuss in the next chapter, page size can significantly impact performance: thus, the IA-32 architecture supports 4KB, 2MB and 4MB pages. When 4KB pages are used, the IA-32 architecture maps each linear address to a' physical address using a two-level page address translation mechanism. In this case, each 32-bit linear address is represented as $V = (t,p, d)$, where t is a 10-bit page table number, p is a 10-bit page number, and d is a 12-bit displacement. Page address translation occurs exactly as described in Section 10.4.4. To locate the page directory entry, t is added to the value of the page directory origin register (analogous to the page table origin register). Each 4-byte **page directory entry (PDE)** specifies *b,* the base address of a page table containing 4-byte page table entries (PTEs). To form the physical address at which the page table entry resides, the processor multiplies p by

4 (because each PTE occupies 4 bytes) and adds *b*. The most active PTEs and PDEs are stored in a processor's TLB to speed virtual-to-physical address translations.

When 4MB pages are used, each 32-bit linear address is represented as *v' - (p, d),* where *p* is a 10-bit page directory number and *d* is a 22-bit displacement. In this case, page address translation occurs exactly as described in Section 10.4.3.

Each page reference is checked against its PDE's and PTE's read/write bit. When the bit is on, all of the pages in the PDE's page table are read-only; otherwise they are read/write. Similarly, each PTE's read/write bit specifies the read/write access right for its corresponding page.

One limitation of a 32-bit physical addresses space is that the system can access at most 4GB of main memory. Because some systems contain more than that, the IA-32 architecture provides the **Physical Address Extension (PAE)** and **Page Size Extension (PSE)** features, which enable a system to reference 36-bit physical addresses, supporting a maximum memory size of 64GB. The implementation of these features is beyond the scope of the discussion; the reader is encouraged to study these and other features of the IA-32 specification, which can be found online in the *IA-32 Intel Architecture Software Developer's Manual, Vol.* 3, located at developer.intel.com/design/Pentium4/manuals/. The Web Resources section contains links to several manuals that describe virtual memory implementations in other architectures, such as the PowerPC and PA-RISC architectures.



Mini Case Study

IBM Mainframe Operating System

The first commercial computer with transistor logic for general use was the IBM 7090 which was released in 1959. Its operating system was called **IBSYS**.[51, 52] IBSYS had roughly two dozen components, which included modules for the FORTRAN and COBOL programming languages, in addition to the resource managers.[53]

In 1964 IBM announced its next set of systems, the System/360 line, which consisted of several different models of varying processor power, all running on the same architecture.[54,55] The smallest models ran under the Disk Operating System or DOS, while the rest came with OS/360.[56, 57, 58] Having one operating system run on several different models of computer was a relatively new concept at the time.[59] OS/360 had three different control options: PCP, MFT, and MVT.[60,61] OS/360-PCP (for Principal Control Program) was single-tasking and designed for the smallest System/360 models; however, in practice DOS (or TOS for systems with tape drives only) was used

(continued)

Mini Case Study

IBM Mainframe Operating Systems (Cont.)

on these computers.[62, 63] OS/360-MFT and OS/360-MVT were both multitasking, with a "Fixed number of Tasks" and a "Variable number of Tasks," respectively, which meant that a system running the MFT option had its memory divisions preset by the operator, while under MVT memory could be divided by the system automatically as new jobs arrived.[64, 65] A large project called TSS (Time-Sharing System) to build a multiuser operating system to compete with Multics was eventually cancelled when TSS proved to be too large and to crash too often.[66] Its functionality was implemented as TSO (the Time-Sharing Option) in 1971.[67, 68]

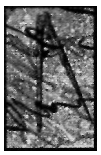
IBM announced the System/370 hardware in 1970, which now included virtual memory support.[69] To utilize this new capability, OS/360-MFT was updated and renamed OSA/S1, while the MVT option was similarly upgraded and named OS/VS2 SVS (Single Virtual Storage), so named because it had only one 16MB virtual address space shared among all users.[70, 71] It was later updated again to allow any number of such address spaces, and so was renamed 0S/VS2 MVS, or simply **MVS** (for Multiple Virtual Storages).[72, 73]

MVS was updated to MVS/370, which soon supported the new cross-memory feature of the System/370 hardware; cross-memory is a mechanism to separate data and code in memory space.[74-75] Additionally, because 16MB of memory space had quickly been filled by expanding system programs, the processors now used a 26-bit address, allowing for 64MB of real memory; however, while MVS/370 supported this real memory capacity, it still used a 24-bit address and therefore only used 16MB virtual memory spaces.[76-77]

Two years later, IBM moved on to the 3081 processors running on the 370-XA "Extended Architecture."[78-79] MVS/370 was updated, new functionality was added, and the resulting package was subsequently released as MVS/XA in 1983.[80-81] This OS now allowed for 2GB of virtual memory, much more easily satisfying any program's space requirements.[82]

In 1988 IBM upgraded the hardware yet again, moving to the ES/3090 processors on the ESA/370 (Enterprise System Architecture).[83] ESA introduced memory spaces that were used only for data, making it easier to move data around for processing.[84] MVS/XA was moved up to MVS/ESA for these new systems.[85-86] MVS/ESA was significantly updated in 1991 for the new ESA/390 hardware, which included mechanisms for linking IBM machines into a loose cluster (SYSPLEX)[87] and changing the settings on I/O devices without the need to take the machine offline (ESCON).[88]

MVS is still the root of IBM's mainframe operating systems, as MVS/ESA was upgraded to OS/390 in 1996 for the System/390 hardware;[89] it is now named **z/OS** and runs on the zSeries mainframes.[90] In addition, IBM's mainframe operating systems have remained backward compatible with earlier versions; OS/390 can run applications written for MVS/ESA, MVS/XA, OSA/S2, or even OS/360.[91]



Mini Case Study

Early History of the VM Operating System

A virtual machine is an illusion of a real machine. 'It is created by a **virtual machine operating system,** which makes a single real machine appear to be several real machines. From the user's viewpoint, virtual machines can appear to be existing real machines, or they can be dramatically different. The concept has proven valuable, and many virtual machine operating systems have been developed—one of the most widely used is IBM's VM.

VM managed the IBM System/370[92]- [93] computer (or compatible hardware), creating the illusion that each of several users had a complete System/370, including many input/output devices. Each user could choose a different operating system. VM can run several operating systems simultaneously, each of them on its own virtual machine. Today's VM users can run versions of the operating systems OS/390, z/OS, TPF, VSE/ESA, CMS and Linux.[94]

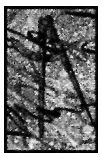
Conventional multiprogramming systems share the resources of a single computer among several processes. These processes are each allocated a portion of the real machine's resources. Each process sees a machine smaller in size and capabilities than the real machine executing the process.

Virtual machine multiprogramming systems (Fig. 10.32) share the resources of a single machine in a different manner. They create the illusion that one real machine is actually several machines. They create virtual processors, virtual storage and virtual I/O devices, possibly with much larger capacities than those of the underlying real machine.

The main components of VM are the **Control Program (CP),** the **Conversational Monitor System (CMS),** the **Remote Spooling Communications Subsystem (RSCS),** the **Interactive Problem Control System (IPCS)** and the **CMS Batch Facility.** CP creates the environment in which virtual machines can execute. It provides the support for the various operating systems normally used to control System/370 and compatible computer systems. CP manages the real machine underlying the virtual machine environment. It gives each user access to the facilities of the real machine such as the processor, storage and I/O devices. CP multiprograms complete virtual machines rather than individual tasks or processes. CMS is an applications system with powerful features for interactive development of programs. It contains editors, language translators, various applications packages and debugging tools. Virtual machines running under CP perform much as if they were real machines, except that they operate more slowly, since VM runs many virtual machines simultaneously.

VM's ability to run multiple operating systems simultaneously has many applications.[95, 96] It eases migration between different operating systems or different versions of the same operating system. It enables people to be trained simultaneously with running production systems. System development can occur simultaneously with production runs. Customers can run different operating systems simultaneously to exploit each system's benefits. Running multiple operating systems offers a form of backup in case of failure; this increases availability. Installations may run benchmarks on new operating

(continued)



Mini Case Study

Early History of the VM Operating System (Cont.)

systems to determine if upgrades are worthwhile; this may be done in parallel with production activity.

Processes running on a virtual machine are not controlled by CP, but rather by the actual operating system running on that virtual machine. The operating systems running on virtual machines perform their full range of functions, including storage management, processor scheduling, input/output control, protecting users from one another, protecting the operating system from the users, spooling, multiprogramming, job and process control, error handling, and so on.

VM simulates an entire machine dedicated to one user. A user at a VM virtual machine sees the equivalent of a complete real machine, rather than many users sharing one set of resources.

CP/CMS began as an experimental system in 1964. It was to be a second-generation timesharing system based on IBM System/360 computers.[97] Originally developed for local use at the IBM Cambridge Scientific Center, it soon gained favor as a tool for evaluating the performance of other operating systems.

The first operational version appeared in 1966, consisting of CP-40 and CMS. These components were designed to run on a modified IBM 360/40 with newly incorporated dynamic address translation hardware. At about this time, IBM announced an upgrade to its powerful 360/65. The new system, the 360/67, incorporated dynamic address translation hardware and provided the basis for a general-purpose timesharing, multiprogramming computer utility called TSS/360.[98] The TSS effort, performed independently of the work on CP/CMS, encountered many difficulties (typical of the large-scale software efforts of the mid-1960s). Meanwhile CP/CMS was successfully moved to the 360/67, and eventually it superseded the TSS effort. CP/CMS evolved into VM/370, which became available in 1972 for the virtual storage models of the IBM 370 series."

CTSS, successfully used at MIT through 1974, most strongly influenced the design of CP/CMS. The CP/CMS designers found that the CTSS design was difficult to work with. They felt that a more modular approach would be appropriate, so they split the resource management portion from the user support portion of the operating system, resulting in CP and CMS, respectively. CP provides separate computing environments that give each user complete access to a full virtual machine; CMS runs on a CP-created virtual machine as a single-user interactive system.

The most significant decision made in the design of CP was that each virtual machine would replicate a real machine. It was clear that the 360 family concept would create a long lifetime for 360 programs; users requiring more power would simply move up in the family to a compatible system with more storage, devices and processor speed. Any decision to produce virtual machines different in structure from a real 360 probably would have resulted in the failure of the CP/CMS system. Instead, the'concept has been successful, making VM one of IBM's two leading mainframe operating systems in the 1990s. IBM continues to create new versions of VM for use on its mainframe servers, the latest being zA/M, which supports IBM's 64-bit architecture.[100]

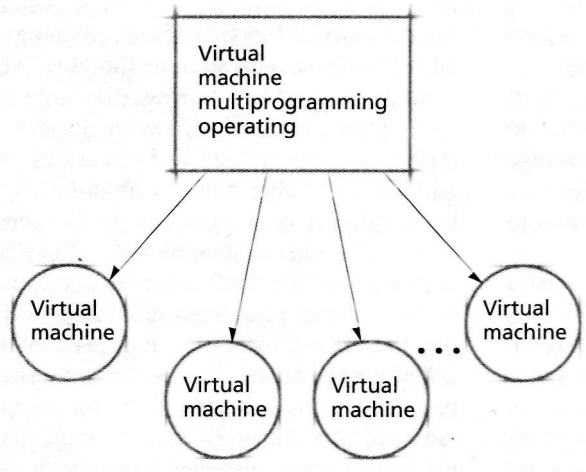


Figure 10.32 | *Virtual machine multiprogramming.*

Web Resources

www.redhat.com/docs/manuals/linux/RHL-8.O-Manual/admin-primer/s1-memory-concepts.html
Overview of virtual memory from Red Hat (a Linux vendor).

www.howstuffworks.com/virtual-memory.htm
Overview of virtual memory.

www.memorymanagement.org
Provides a glossary and guide to physical and virtual memory management.

developer.intel.com
Provides technical documentation regarding Intel products, including Pentium and Itanium processors. Their system programmer guides (e.g., for the Pentium 4 processor, developer.intel.com/design/Pentium4/manuals/) provide information on how memory is organized and accessed in Intel systerns.

www-3.ibm.com/chips/techlib/techlib.nsf/product-families/PowerPC_Microprocessors_and_Embedded_Processors
Provides technical documentation regarding PowerPC processors, found in Apple computers. Descriptions of virtual memory management in these processors is located in the software reference manuals (e.g., the PowerPC 970, found in Apple G5 computers, www-3.ibm.com/chips/techlib/techlib.nsf/products/PowerPC_970_Microprocessor).

h21007.www2.hp.com/dspp/tech/tech_TechDocumentDetailPage_IDX/1,1701,958!13!253,00.html
Describes virtual memory organization for PA-RISC processors. PA-RISC is primarily used in high-performance environments. See also cpus.hp.com/technical_references/pari´sc.shtml.

Summary

Virtual memory solves the problem of limited memory space by creating the illusion that more memory exists than is available in the system. There are two types of addresses in virtual memory systems: those referenced by processes and those available in main memory. The addresses that processes reference are called virtual addresses. The addresses available in main memory are called physical (or real) addresses. Whenever a process accesses a virtual address, it must translate the virtual addresses to a physical address; this is done by the memory management unit (MMU).

A process's virtual address space, V, is the range of virtual addresses that it may reference. The range of physical addresses available on a particular computer system is called that computer's real address space, R. If we are to

permit a user's virtual address space to be larger than its real address space, we must provide a means for retaining programs and data in a large auxiliary storage. A system normally accomplishes this by employing a two-level storage scheme consisting of main memory and secondary storage. When the system is ready to run a process, the system loads the process's code and data from secondary storage into main memory. Only a small portion of a process's code and data needs to be in main memory for that process to execute.

Dynamic address translation (DAT) mechanisms convert virtual addresses to physical addresses during execution. Systems that use DAT exhibit the property that the contiguous addresses in a process's virtual address space need not be contiguous in physical memory—this is called artificial contiguity. Dynamic address translation and artificial contiguity free the programmer from concerns about memory placement (e.g., the programmer need not create overlays to ensure the system can execute the program). Dynamic address translation mechanisms must maintain address translation maps indicating which regions of a process's virtual address space, V, are currently in main memory, and where they are located.

Virtual memory systems cannot afford to map addresses individually, so information is grouped into blocks, and the system keeps track of where in main memory the various virtual memory blocks have been placed. When blocks are the same size, they are called pages and the associated virtual memory organization technique is called paging. When blocks may be of different sizes, they are called segments, and the associated virtual memory organization technique is called segmentation. Some systems combine the two techniques, implementing segments as variable-size blocks composed of fixed-size pages.

In a virtual memory system with block mapping, the system represents addresses as ordered pairs. To refer to a particular item in the process's virtual address space, the process specifies the block in which the item resides, and the displacement (or offset) of the item from the start of the block. A virtual address, v, is denoted by an ordered pair *(b, d),* where *b* is the block number in which the referenced item resides, and *d* is the displacement from the start of the block. The system maintains in memory a block map table for each process. The real address, a, that corresponds to the address in main memory of a process's block map table is loaded into a special processor register called the block map table origin register. During execution, the process references a virtual address *v = (b, d).The* system adds the block number, *b,* to the base address, *a,* of the process's block table to form the real address of the entry for block *b* in the block map table. This entry contains the address, *b′.* for the start of block *b* in main memory. The system then adds the displacement, *d,* to the block start address, *b′,* to form the desired real address, *r.*

Paging uses a fixed-size block mapping; pure paging systems do not combine segmentation with paging. A virtual address in a paging system is an ordered pair *(p, d),* where *p* is the number of the page in virtual memory on which the referenced item resides, and *d* is the displacement within page *p* at which the referenced item is located. When the system transfers a page from secondary storage to main memory, the system places the page in a main memory block, called a page frame, that is the same size as the incoming page. Page frames begin at physical memory addresses that are integral multiples of the fixed page size. The system may place an incoming page in any available page frame.

Dynamic address translation under paging is similar to block address translation. A running process references a virtual memory address *v = (p,d).A* page mapping mechanism uses the value of the page table origin register to locate the entry for page *p* in the process's page map table (often simply called the page table). The corresponding page table entry (PTE) indicates that page *p* is in page frame *p′ (p′* is not a physical memory address). The real memory address, *r,* corresponding to v is formed by concatenating *p′* and the displacement into the page frame, *d.* which places *p′* in the most-significant bits of the real memory address and *d* in the least-significant bits.

When a process references a page that is not in main memory, the processor generates a page fault, which invokes the operating system to load the missing page into memory from secondary storage. In the PTE for this page, a resident bit, *r,* is set to 0 if the page is not in main memory and 1 if the page is in main memory.

Page address translation can be performed by direct mapping, associative mapping or combined direct/associative mapping. Due to the cost of high-speed, location-addressed cache memory and the relatively large size of programs, maintaining the entire page table in cache memory is often not viable, which limits the performance of direct-mapping page address translation because the page table must then be stored in much slower main memory. One way to increase the performance of dynamic address translation is to place the entire page table into a content-addressed (rather than location-addressed) associative memory, which has a cycle time perhaps an order of magnitude faster than main memory. In this case, every entry in the associative memory is searched simultaneously for page

p. In the vast majority of systems, using a cache memory to implement pure direct mapping or an associative memory to implement pure associative mapping is too costly.

As a result, many designers have chosen a compromise scheme that offers many of the advantages of the cache or associative memory approach but at a more modest cost. This approach uses an associative memory, called the translation lookaside buffer (TLB), capable of holding only a small percentage of the complete page table for a process. The page table entries maintained in this map typically correspond to the rhost-recently referenced pages only, using the heuristic that a page referenced recently in the past is likely to be referenced again in the near future. This is an example of locality (more specifically of temporal locality —locality in time). When the system locates the mapping for *p* in the TLB, it experiences a TLB hit; otherwise a TLB miss occurs, requiring the system to access slower main memory. Empirically, due to the phenomenon of locality, the number of entries in the TLB does not need to be large to achieve good performance.

Multilevel (or hierarchical) page tables enable the system to store in discontiguous locations in main memory those portions of a process's page table that the process is using. These layers form a hierarchy of page tables, each level containing a table that stores pointers to tables in the level below. The bottom-most level is comprised of tables containing address translations. Multilevel page tables can reduce memory overhead compared to a direct-mapping system, at the cost of additional accesses to main memory to perform address translations that are not contained in the TLB.

An inverted page table stores exactly one PTE in memory for each page frame in the system. The page tables are inverted relative to traditional page tables because the PTEs are indexed by page frame number rather than virtual page number. Inverted page tables use hash functions and chaining pointers to map a virtual page to an inverted page table entry, which yields a page frame number. The overhead incurred by accessing main memory to follow each pointer in an inverted page table's hash collision chain can be substantial. Because the size of the inverted page table must remain fixed to provide direct mappings to page frames, most systems employ a hash anchor table that increases the range of the hash function to reduce collisions, at the cost of an additional memory access.

Enabling sharing in multiprogramming systems reduces the memory consumed by programs that use common data and/or instructions, but requires that the system identify each page as either sharable or nonsharable. If the page table entries of different processes point to the same page frame, the page frame is shared by each of the processes. Copy-on-write uses shared memory to reduce process creation time by sharing a parent's address space with its child. Each time a process writes to a shared page, the operating system copies the shared page to a new page that is mapped to that process's address space. The operating system can mark these shared pages as read-only so that the system generates an exception when a process attempts to modify a page. To handle the exception, the operating system allocates an unshared copy of the page to the process that generated the exception. Copy-on-write can result in poor performance if the majority of a process's shared data is modified during program execution.

Under segmentation, a program's data and instructions are divided into blocks called segments. Each segment contains a meaningful portion of the program, such as a procedure or an array. Each segment consists of contiguous locations; however, the segments need not be the same size nor must they be adjacent to one another in main memory. A process may execute while its current instructions and referenced data are in segments in main memory. If a process references memory in a segment not currently resident in main memory, the virtual memory system must retrieve that segment from secondary storage. The placement strategies for segmentation are identical to those used in variable-partition multiprogramming.

A virtual memory segmentation address is an ordered pair $v = (s, d)$, where s is the segment number in which the referenced item resides, and d is the displacement within segment s at which the referenced item is located. The system adds the segment number, s, to the segment map table's base address value, b, located in the segment map table origin register. The resulting value, $b + s$, is the location of the segment map table entry. If the segment currently resides in main memory, the segment map table entry contains the segment's physical memory address, s'. The system then adds the displacement, d, to this address to form the referenced location's real memory address, $r = s' + d$. We cannot simply concatenate s' and d, as we could in a paging system, because segments are of variable size. If the segment is not in main memory, then a missing-segment fault is generated, causing the operating system to gain control and load the referenced segment from secondary storage.

Each reference to a segment is checked against the segment length, /, to ensure that it falls within the segment's range. If it does not, then a segment-overflow exception is generated, causing the operating system to gain control and terminate the process. Each reference to the

segment is also checked against the protection bits in the segment map table entry to determine if the operation is allowed. If it is not, then a segment-protection exception is generated, causing the operating system to gain control and terminate the process. Sharing segments can incur less overhead than sharing in a direct-mapped pure paging system, because sharing information for each segment (which may consume several pages of memory) is maintained in one segment map table entry.

One scheme for implementing memory protection in segmentation systems is the use of memory protection keys. In this case, each process is associated with a value, called a protection key. When the process references a particular segment, it checks the protection key of the block containing the referenced item. If the protection key for the processor and the requested block are the same, the process can access the segment. The operating system can exercise further protection control by specifying how a segment may be accessed and by which processes. This is accomplished by assigning each process certain access rights, such as read, write, execute and append. By either allowing or denying each of these access types, it is possible to create different access control modes.

In a combined segmentation/paging system, segments occupy one or more pages. All the pages of a segment need not be in main memory at once, and pages contiguous in virtual memory need not be contiguous in main memory. Under this scheme, a virtual memory address is implemented as an ordered triple v = *(s,p, d),* where *s* is the segment number, *p* the page number within the segment and *d* is the displacement within the page at which the desired item is located. When a process references a virtual address *v = (s,p,d),* the system attempts to find the corresponding page frame number *p′* in the TLB. If it is not present, the system first searches the segment map table, which points to the base of a page table, then uses the page number *p* as the displacement into the page table to locate the PTE containing the page frame number *p′*. The displacement, *d,* is then concatenated to *p′* to form the real memory address. This translation scheme assumes that the process has made a valid memory reference and that every piece of information required for the process is located in main memory. Under many conditions however, the address translation may require extra steps or may fail.

In segmentation/paging systems, two processes share memory when each process has a segment map table entry that points to the same page table. Sharing, whether in paged systems, segmented systems, or segmentation/paging systems, requires careful management by the operating system.

The IA-32 Intel architecture supports either pure segmentation or segmentation/paging virtual memory. The set of addresses contained in each segment is called a logical address space; segments are placed in any available location in the system's linear address space, which is a 32-bit (i.e., 4GB) virtual address space. Under pure segmentation, a referenced item's linear address is its address in main memory. If paging is enabled, the linear address space is divided into fixed-size page frames that are mapped to main memory. Segment address translation is performed by a direct mapping that uses high-speed processor registers to store segment map table origin registers in the global descriptor table register (GDTR) or in the local descriptor table register (TDTR), depending on the number of segment map tables in the system. The processor also stores segment numbers called segment descriptors in high-speed registers to improve address translation performance. Under segmentation/paging, segments are placed in the 4GB linear address space, which is divided into page frames that map to main memory. The IA-32 architecture uses a two-level page table to perform translations from page numbers to page frame numbers, which are concatenated with the page offset to form a real address.

Key Terms

access control mode—Set of privileges (e.g., read, write, execute and/or append) that determine how a page or segment of memory can be accessed.

access right—Privilege that determines which resources a process may access. In memory, access rights determine which pages or segments a process can access, and in what manner (i.e., read, write, execute and/or append).

address translation map—Table that assists in the mapping of virtual addresses to their corresponding real memory addresses.

append access—Access right that enables a process to write additional information at the end of a segment but not to modify its existing contents; see also execute access, read access and write access.

artificial contiguity—Technique employed by virtual memory systems to provide the illusion that a program's instructions and data are stored contiguously when pieces of them may be spread throughout main memory; this simplifies programming.

associative mapping—Content-addressed associative memory that assists in the mapping of virtual addresses to their corresponding real memory addresses; all entries of the associative memory are searched simultaneously.

associative memory—Memory that is searched by content, not by location; fast associative memories can help implement high-speed dynamic address translation mechanisms.

block—Portion of a memory space (either real or virtual) defined by a range of contiguous addresses.

block map table—Table containing entries that map each of a process's virtual blocks to a corresponding block in main memory (if there is one). Blocks in a virtual memory system are either segments or pages.

block map table origin register—Register that stores the address in main memory of a process's block map table; this high-speed register facilitates rapid virtual address translation.

block mapping—Mechanism that, a virtual memory system, reduces the number of mappings between virtual memory addresses and real memory addresses by mapping blocks in virtual memory to blocks in main memory.

chaining (hash tables)—Technique that resolves collisions in a hash table by placing each unique item in a data structure (typically a linked list). The position in the hash table at which the collision occurred contains a pointer to that data structure.

CMS Batch Facility (VM)-VM Component that allows the user to run longer jobs in a separate virtual machine so that the user can continue interactive work.

virtual machine operating system—Software that creates the virtual machine.

collision (hash tables)—Event that occurs when a hash function maps two different items to the same position in a hash table. Some hash tables use chaining to resolve collisions.

Conversational Monitor System (CMS) (VM)—Component of VM that is an interactive application development environment.

Control Program (CP) (VM)—Component of VM that runs the physical machine and creates the environment for the virtual machine.

copy-on-write—Mechanism that improves process creation efficiency by sharing mapping information between parent and child until a process modifies a page, at which point a new copy of the page is created and allocated to that process. This can incur substantial overhead if the parent or child modifies many of the shared pages.

direct mapping—Address translation mechanism that assists in the mapping of virtual addresses to their corresponding real addresses, using an index into a table stored in location-addressed memory.

displacement—Distance of an address from the start of a block, page or segment, also called offset.

dynamic address translation (DAT)—Mechanism that converts virtual addresses to physical addresses during execution; this is done at extremely high speed to avoid slowing execution.

execute access—Access right that enables a process to execute instructions from a page or segment; see also read access, write access and append access.

general protection fault (GPF) (IA-32 Intel architecture)—Occurs when a process references a segment to which it does not have appropriate access rights or references an address outside of the segment.

global descriptor table (GDT) (IA-32 Intel architecture)—Segment map table that contains mapping information for process segments or local descriptor table (LDT) segments, which contain mapping information for process segments.

granularity bit (IA-32 Intel architecture)—Bit that determines how the processor interprets the size of each segment, specified by the 20-bit segment limit. When the bit is off, segments range in size from 1 byte to 1MB, in 1-byte increments. When the bit is on, segments range in size from 4KB to 4GB, in 4KB increments.

hash anchor table—Hash table that points to entries in an inverted page table. Increasing the size of the hash anchor table decreases the number of collisions, which improves the speed of address translation, at the cost of the increased memory overhead required to store the table.

hash function—Function that takes a number as input and returns a number, called a hash value, within a specified range. Hash functions facilitate rapidly storing and retrieving information from hash tables.

hash value—Value returned by a hash function that corresponds to a position in a hash table.

hash table—Data structure that indexes items according to their hash values; used with hash functions to rapidly store and retrieve information.

IBSYS—Operating system for the IBM 7090 mainframe.

Interactive Problem Control System (IPCS) (VM)-VM component that provides online analysis and correction of VM software problems.

inverted page table—Page table containing one entry for each page frame in main memory. Inverted page tables incur less table fragmentation than traditional page tables, which typically maintain in memory a greater number of

page table entries than page frames. Hash functions map virtual page numbers to an index in the inverted page table.

linear address space (IA-32 Intel architecture)—32-bit (4GB) virtual address space. Under pure segmentation, this address space is mapped directly to main memory. Under segmentation/paging, this address space is divided into page frames that are mapped to main memory.

local descriptor table (LDT) (IA-32 Intel architecture)—Segment map table that contains mapping information for process segments. The system may contain up to 8,191 LDTs, each containing 8,192 entries.

locality—Empirical phenomenon describing events that are closely related in space or time. When applied to memory access patterns, spatial locality states that when a process references a particular address, it is also likely to access nearby addresses; temporal locality states that when a process references a particular address, it is likely to reference it again soon.

logical address space (IA-32 Intel architecture)—Set of addresses contained in a segment.

Memory Management Unit (MMU)—Special-purpose hardware that performs virtual-to-physical address translation.

missing-segment fault—Fault that occurs when a process references a segment that is not currently in main memory. The operating system responds by loading the segment from secondary storage into main memory when space is available.

multilevel paging system—Technique that enables the system to store portions of a process's page table in discontiguous locations in main memory and store only those portions that a process is actively using. Multilevel page tables are implemented by creating a hierarchy of page tables, each level containing a table that stores pointers to tables in the level below. The bottom-most level is comprised of tables containing the page-to-page-frame mappings. This reduces memory waste compared to single-level page tables, but incurs greater overhead due to the increased number of memory accesses required to perform address translation when corresponding mappings are not contained in the TLB.

Multiple Virtual Spaces (MVS)—IBM operating system for System/370 mainframes allowing any number of 16MB virtual address spaces.

offset—See displacement.

OS/360—Operating system for the IBM System/360 mainframes. OS/360 had two major options, MFT and MVT, which stood for "Multiprogramming with a Fixed number of Tasks" and "Multiprogramming with a Variable number of Tasks." OS/360-MVT evolved into MVS, the ancestor of the current IBM mainframe operating system z/OS.

page—Fixed-size set of contiguous addresses in a process's virtual address space that is managed as one unit. A page contains portions of a process's data and/or instructions and can be placed in any available page frame in main memory.

page directory entry (IA-32 Intel architecture)—Entry in a page directory that maps to the base address of a page table, which stores page table entries.

page fault—Fault that occurs as the result of an error when a process attempts to access a nonresident page, in which case the operating system can load it from disk.

page frame—Block of main memory that can store a virtual page. In systems with a single page size, any page can be placed in any available page frame.

page global directory—In a two-tiered multilevel page table, the page global directory is a table of pointers to portions of a process's page table. Page global directories are the top level of a multilevel page table hierarchy.

page map table—See page table.

page table origin register—Register that holds the location of a process's page table in main memory; having this information accessible in a high-speed register facilitates rapid virtual-to-physical address translation.

page table—Table that stores entries that map page numbers to page frames. A page table contains an entry for each of a process's virtual pages.

page table entry (PTE)—Entry in a page table that maps a virtual page number to a page frame number. Page table entries store other information about a page, such as how the page may be accessed and whether the page is resident.

paging—Virtual memory organization technique that divides an address space into fixed-size blocks of contiguous addresses. When applied to a process's virtual address space, the blocks are called pages, which store process data and instructions. When applied to main memory, the blocks are called page frames. A page is stored on secondary storage and loaded into a page frame if one is available. Paging trivializes the memory placement decision and does not incur external fragmentation (for systems that contain a single page size); paging does incur internal fragmentation.

physical address—Location in main memory.

Physical Address Extension (PAE) (IA-32 Intel architecture)—Mechanism that enables IA-32 processors to address up to 64GB of main memory.

physical address space—Range of physical addresses corresponding to the size of main memory in a given computer. The physical address space may be (and is often) smaller than each process's virtual address space.

process **table**—Table of known processes. In a segmentation/paging system, each entry points to a process's virtual address space, among other items. '

pure paging—Memory organization technique that employs paging only, not segmentation.

read access—Access right that enables a process to read data from a page or segment; see also execute access, write access and append access.

real address—See physical address.

Remote Spooling Communications Subsystem (RSCS) (VM) — Component of VM that provides the capability to send and receive files in a distributed system.

segment—Variable-size set of contiguous addresses in a process's virtual address space that is managed as one unit. A segment is typically the size of an entire set of similar items, such as a set of instructions in a procedure or the contents of an array, which enables the system to protect such items with fine granularity using appropriate access rights. For example, a data segment typically is assigned read-only or read/write access, but not execute access. Similarly, a segment containing executable instructions typically is assigned read/execute access, but not write access. Segments tend to create external fragmentation in main memory but do not suffer from internal fragmentation.

segment descriptor (IA-32 Intel architecture) — Segment map table entry that stores a segment's base address, present bit, limit address and protection bits.

segment map table origin register—Register that holds the location of a process's segment map table in main memory; having this information accessible in a high-speed register facilitates rapid virtual-to-physical address translation.

segment-overflow exception—Exception that occurs when a process attempts to access an address that is outside a segment.

segment-protection exception—Exception that occurs when a process attempts to access a segment in ways other than those specified by its access control mode (e.g., attempting to write to a read-only segment).

segment **selector** (IA-32 Intel architecture) — 16-bit value indicating the offset into the segment map table at which the corresponding segment descriptor (i.e., segment map table entry) is located.

table fragmentation—Wasted memory consumed by block mapping tables; small blocks tend to increase the number of blocks in the system, which increases table fragmentation.

translation lookaside buffer (TLB)—High-speed associative memory map that holds a small number of mappings between virtual page numbers and their corresponding page frame numbers. The TLB typically stores recently used page table entries, which improves performance for processes exhibiting locality.

virtual address—Address that a process accesses in a virtual memory system; virtual addresses are translated to real addresses dynamically at execution time.

virtual address space—Range of virtual addresses that a process may reference.

virtual memory—Technique that solves the problem of limited memory by providing each process with a virtual address space (potentially larger than the system's physical address space) that the process uses to access data and instructions.

write access—Access right that enables a process to modify the contents of a page or segment; see also execute access, read access and append access.

z/OS—IBM operating system for zSeries mainframes and the latest version of MVS.

Exercises

10.1 Give several reasons why it is useful to separate a process's virtual memory space from its physical memory space.

10.2 One attraction of virtual memory is that users no longer have to restrict the size of their programs to make them fit into physical memory. Programming style becomes a freer form of expression. Discuss the effects of such a free programming style on performance in a multiprogramming virtual memory environment. List both positive and negative effects.

10.3 Explain the various techniques used for mapping virtual addresses to physical addresses under paging.

10.4 Discuss the relative merits of each of the following virtual memory mapping techniques.

a. direct mapping

b. associative mapping

c. combined direct/associative mapping

10.5 Explain the mapping of virtual addresses to physical addresses under segmentation.

10.6 Consider a pure paging system that uses 32-bit addresses (each of which specifies one byte of memory), contains 128MB of main memory and has a page size of 8KB.

a. How many page frames does the system contain?

b. How many bits does the system use to maintain the displacement, *d*?

c. How many bits does the system use to maintain the page number,*p*?

10.7 Consider a pure paging system that uses three levels of page tables and 64-bit addresses. Each virtual address is the ordered set $v = (p,m, t, d)$, where the ordered triple (p, m, t) is the page number and d is the displacement into the page. Each page table entry is 64 bits (8 bytes). The number of bits that store p is n_p, the number of bits that store m is n_m and the number of bits to store t is n_t.

a. Assume $n_p = n_m = n_t = 18$.

i. How large is the table at each level of the multi-level page table?

ii. What is the page size, in bytes?

b. Assume $n_p = n_m = n_t = 14$.

i. How large the table at each level of the multi-level page table?

ii. What is the page size, in bytes?

c. Discuss the trade-offs of large and small table sizes.

10.8 Explain how memory protection is implemented in virtual memory systems with segmentation.

10.9 Discuss the various hardware features useful for implementing virtual memory systems.

10.10 Discuss how fragmentation manifests itself in each of the following types of virtual memory systems.

a. segmentation

b. paging

c. combined segmentation/paging

10.11 In any computer system, regardless of whether it is a real memory system or a virtual memory system, the computer will rarely reference all the instructions or data brought into main memory. Let us call this *chunk fragmentation,* because it is the result of handling memory items in blocks or chunks rather than individually. Chunk fragmentation might actually account for more waste of main memory than all other types of fragmentation combined.

a. Why, then, has chunk fragmentation not been given the same coverage in the literature as other forms of fragmentation?

b. How do virtual memory systems with dynamic memory allocation greatly reduce the amount of chunk fragmentation over that experienced in real memory systems?

c. What effect would smaller page sizes have on chunk fragmentation?

d. What considerations, both practical and theoretical, prevent the complete elimination of chunk fragmentation?

e. What can each of the following do to minimize chunk fragmentation?

i. the programmer

ii. the hardware designer

iii. the operating system designer

10.12 Explain the mapping of virtual addresses to physical addresses under combined segmentation/paging.

10.13 In multiprogramming environments, code and data sharing can greatly reduce the main memory needed by a group of processes to run efficiently. For each of the following types of systems, outline briefly how sharing can be implemented.

a. fixed-partition multiprogramming

b. variable-partition multiprogramming

c. paging

d. segmentation

e. combined segmentation/paging

10.14 Why is sharing of code and data so much more natural in virtual memory systems than in real memory systems?

10.15 Discuss the similarities and differences between paging and segmentation.

10.16 Compare and contrast pure segmentation with segmentation/paging combined.

10.17 Suppose you are asked to implement segmentation on a machine that has paging hardware but no segmentation hardware. You may use only software techniques. Is this possible? Explain your answer.

10.18 Suppose you are asked to implement paging on a machine that has segmentation hardware but no paging hardware. You may use only software techniques. Is this possible? Explain your answer.

10.19 As the chief designer of a new virtual memory system you have been given the choice of implementing either paging or segmentation, but not both. Which would you choose? Why?

10.20 Suppose that economical associative memory were to become available. How might such associative memory be incorporated into future computer architectures to improve

the performance of the hardware, the operating system, and user programs?

10.21 Give as many ways as you can in which executing a program on a virtual memory system differs from executing the same program on a real memory system. Do these observations lead you to favor real memory approaches or virtual memory approaches?

10.22 Give as many reasons as you can why locality is a reasonable phenomenon. Give as many examples as you can of situations in which locality simply does not apply.

10.23 One popular operating system provides separate virtual address spaces to each of its processes, while another has all processes share a single large address space. Compare and contrast these two different approaches.

10.24 Virtual address translation mechanisms are not without their costs. List as many factors as you can that contribute to the overhead of operating a virtual-address-to-physical-address translation mechanism. How do these factors tend to "shape" the hardware and software of systems that support such address translation mechanisms?

10.25 The Multics system, as originally designed, provided for two page sizes. It was believed that the large number of small data structures could occupy small pages, and that most other procedures and data structures would best occupy one or more large pages. How does a memory organization scheme that supports multiple page sizes differ from one that supports a single page size? Suppose a system were designed to support n page sizes. How would it differ from the Multics approach? Is a system that supports a large number of page sizes essentially equivalent to a segmentation system? Explain.

10.26 With multilevel page tables, a process's PTE can be initialized when the process is first loaded, or each PTE can be initialized the first time its corresponding page is referenced. Similarly, the entire multilevel page table structure can be maintained in main memory, or parts of it can be sent to secondary storage. What are the costs and benefits of each approach?

10.27 Why did virtual memory emerge as an important scheme? Why did real memory schemes prove inadequate? What current trends could conceivably negate the usefulness of virtual memory?

10.28 What aspect of paging with a single page size makes page-replacement algorithms so much simpler than segment-replacement algorithms? What hardware or software features might be used in support of segment replacement that could make it almost as straightforward as page replacement in a system with a single page size?

10.29 What aspect of content-addressed associative memory will probably ensure that such memory will remain far more costly than location-addressed cache memory?

10.30 Suppose you are designing a portion of an operating system that requires high-speed information retrieval (such as a virtual address translation mechanism). Suppose that you have been given the option of implementing your search-and-retrieval mechanism with either pure direct mapping, using a high-speed cache, or pure associative mapping. What factors would influence your choice? What interesting kind(s) of question(s) could conceivably be answered by an associative search (in one access) that cannot be answered in a single direct-mapped search?

10.31 Deciding what entries to keep in the TLB is crucial to the efficient operation of a virtual memory system. The percentage of references that get "resolved" via the TLB is called the TLB hit ratio. Compare and contrast the performance of virtual address translation systems that achieve a high (near 100 percent) hit ratio versus those that achieve a low (near 0 percent) hit ratio. List several heuristics not mentioned in the text that you believe would achieve high hit ratios. Indicate examples of how each might fail, i.e., under what circumstances would these heuristics place the "wrong" entries into the TLB?

Suggested Projects

10.32 Describe how the IA-32 architecture enables processes to access up to 64GB of main memory. See developer.intel.com/design/Pentium4/manuals/.

10.33 Compare and contrast the IBM/Motorola PowerPC architecture virtual memory implementation with that of the IA-32 Intel architecture. Discuss how 64-bit processing affects memory organization in the PowerPC. See developer.intel.com/design/Pentium4/manuals/ and www-3.ibm.com/chips/techlib/techlib.nsf/productfamilies/PowerPC_Microprocessors_and_Embedded_Processors.

10.34 Compare and contrast the 64-bit Hewlett-Packard PA-RISC architecture virtual memory implementation with that of the 32-bit Intel IA-32 architecture. See developer.intel.com/design/Pentium4/manuals/ and cpus.hp.com/technical_references/parisc.shtml.

10.35 Survey the differences between the virtual memory implementation in the 32-bit Intel IA-32 architecture and in the 64-bit IA-64 Intel architecture. See developer.intel.com/design/Pentium4/manuals/ and developer.intel.com/design/itanium/manuals/iiasdmanual.htm (select Volume 2: System Architecture).

Recommended Reading

The concept of virtual memory has been around for decades; an early comprehensive survey was produced by Denning.[101] The benefits of virtual memory systems spurred the development of specialized hardware to increase their performance and feasibility.[102] Associative memories for dynamic address translation were presented by Hanlon,[103] and their role in storing TLB entries has been studied thoroughly.[104]

Denning discusses paging in his work on virtual memory; an in-depth treatment of segmentation is provided by Dennis.[105] Further discussions on virtual memory organization, such as combined segmentation and paging, can be found in papers by Daley and Dennis[106] and Denning.[107] Jacob and Mudge[108] provide an excellent survey of virtual memory organization techniques and their implementations in modern systems. Two popular textbooks on computer architecture that describe the implementations of MMUs are Patterson and Hennessy's *Computer Organization and Design*[109] and Blaauw and Brooks's *Computer Architecture.*[110] The bibliography for this chapter is located on our Web site at www.deitel.com/books/os3e/Bibliography.pdf.

Works Cited

1. Shiell, I, "Virtual Memory, Virtual Machines," *Byte,* Vol. 11, No. 11,1986, pp. 110-121.
2. Leonard, T. E., ed., *VAX Architecture Reference Manual,* Bedford, MA: Digital Press, 1987.
3. Kenah, L. J.; R. E. Goldenberg; and S. F. Bate, *VAX/VMS Internals and Data Structures,* Bedford, MA: Digital Press, 1988.
4. Fotheringham, X, "Dynamic Storage Allocation in the Atlas Computer, Including an Automatic Use of a Backing Store," *Communications of the ACM,* Vol. 4,1961, pp. 435-436.
5. Kilburn, T.; D. J. Howarth; R. B. Payne; and F. H. Sumner, "The Manchester University Atlas Operating System, Part I: Internal Organization," *Computer Journal,* Vol. 4, No. 3, October 1961, pp. 222-225.
6. Kilburn, T.; R. B. Payne; and D. J. Howarth, "The Atlas Supervisor," *Proceedings of the Eastern Joint Computer Conference, AFIPS,Vol.* 20, 1961.
7. Lavington, S. H.. "The Manchester Mark I and Atlas: A Historical Perspective," *Communications of the ACM,* Vol. 21, No. 1, January 1978, pp. 4-12.
8. Manchester University Department of Computer Science, "The Atlas," 1996, <www.computer50.org/kgill/atlas/atlas.html>.
9. Manchester University Department of Computer Science, "History of the Department of Computer Science," December 14, 2001,<www.cs.man.ac.uk/Visitor_subweb/history.php3>.
10. Manchester University Department of Computer Science, "History of the Department of Computer Science," December 14, 2001, <www.cs.man.ac.uk/Visitor_subweb/history.php3>.
11. Lavington, S., "The Manchester Mark 1 and Atlas: A Historical Perspective," *Communications of the ACM,* January 1978, pp. 4-12.
12. Lavington, S., "The Manchester Mark I and Atlas: A Historical Perspective," *Communications of the ACM,* January 1978, pp. 4-12.
13. Manchester University Department of Computer Science, "The Atlas," 1996, <www.computer50.org/kgill/atlas/atlas.html>.
14. Lavington, S., "The Manchester Mark I and Atlas: A Historical Perspective," *Communications of the ACM,* January 1978, pp. 4-12.
15. Lavington, S., "The Manchester Mark I and Atlas: A Historical Perspective," *Communications of the ACM,* January 1978, pp. 4-12.
16. Manchester University Department of Computer Science "The Atlas," 1996, <www.computer50.org/kgill/atlas/atlas. html>.
17. Lavington, S., "The Manchester Mark I and Atlas: A Historical Perspective," *Communications of the ACM,* January 1978, pp. 4-12.
18. Denning, P., "Virtual Memory," *ACM Computing Surveys,* Vol. 2. No. 3, September 1970, pp. 153-189.
19. George Mason University, "Peter J. Denning—Biosketch," January 1,2003, <cne.gmu.edu/pjd/pjdbio.html>.
20. Denning, P., "The Working Set Model for Program Behavior." *Communications of the ACM,* Vol. 11, No. 5, May 1968, pp. 323-333.
21. George Mason University, "Peter J. Denning—Biosketch," January 1,2003, <cne.gmu.edu/pjd/pjdbio.html>.
22. George Mason University, "Peter J. Denning—Biosketch," January 1,2003, <cne.gmu.edu/pjd/pjdbio.html>.
23. ACM, "Peter J. Denning-ACM Karlstrom Citation 1996," 1996. <cne.gmu.edu/pjd/pjdkka96.html>.
24. Randell, B., and C. J. Kuehner, "Dynamic Storage Allocation Systems," *Proceedings of the ACM Symposium on Operating System Principles,* January 1967, pp. 9.1-9.16.
25. McKeag, R. M., "Burroughs B5500 Master Control Program," in *Studies in Operating Systems,* Academic Press, 1976, pp. 1-66.
26. Oliphint, C, "Operating System for the B5000," *Datamation,* Vol. 10, No. 5,1964, pp. 42-54.
27. Talluri, M.; S. Kong; M. D. Hill; and D. A. Patterson, "Tradeoffs in Supporting Two Page Sizes," In *Proceedings of the 19th International Symposium on Computer Architecture,* Gold Coast, Australia, May 1992, pp. 415-424.

28. Hanlon, A. G., "Content-Addressable and Associative Memory Systems—A Survey," *IEEE Transactions on Electronic Computers,* August 1966.
29. Lindquist, A. B.; R. R. Seeder; and L. W. Comeau, "A Time-Sharing System Using an Associative Memory," *Proceedings of the IEEE,* Vol. 54,1966, pp. 1774-1779.
30. Cook, R., et al., "Cache Memories: A Tutorial and Survey of Current Research Directions," *ACM/CSC-ER,* 1982, pp. 99-110.
31. Wang, Z.; D. Burger; K. S. McKinley; S. K. Reinhardt; and C. C. Weems, "Guided Region Prefetching: A Cooperative Hardware/ Software Approach," *Proceedings of the 30th Annual International Symposium on Computer Architecture,* 2003, p. 388.
32. Jacob, B. L., and T. N. Mudge, "A Look at Several Memory Management Units, TLB-Refill Mechanisms, and Page Table Organizations," *Proceedings of the Eighth International Conference on Architectural Support for Programming Languages and Operating Systems,* 1998, pp. 295-306.
33. Kandiraju, G. B., and A. Sivasubramaniam, "Characterizing the d-TLB Behavior of SPEC CPU2000 Benchmarks," ACM SIGMETRICS Performance Evaluation Review, *Proceedings of the 2002 ACM SIGMETRICS International Conference on Measurement and Modeling of Computer Systems,* Vol. 30, No. 1, June 2002.
34. Sohoni, S.; R. Min; Z. Xu; and Y. Hu, "A Study of Memory System Performance of Multimedia Applications," ACM SIGMETRICS Performance Evaluation Review, *Proceedings of the 2001 ACM SIGMETRICS International Conference on Measurement and Modeling of Computer Systems,Vol.* 29, No. 1, June 2001.
35. Kandiraju, Gokul B., and Anand Sivasubramaniam, "Going the Distance for TLB Prefetching: An Application-Driven Study," ACM SIGARCH Computer Architecture News, *Proceedings of the 29th Annual International Symposium on Computer Architecture,* Vol. 30, No. 2, May 2002.
36. Jacob, B., and T. Mudge, "Virtual Memory: Issues of Implementation," *IEEE Computer,* Vol. 31, No. 6, June 1998, p. 36.
37. Jacob, B., and T. Mudge, "Virtual Memory: Issues of Implementation," *IEEE Computer,* Vol. 31, No. 6, June 1998, pp. 37-38.
38. Shyu, I., "Virtual Address Translation for Wide-Address Architectures," *ACM SIGOPS Operating Systems Review,* Vol. 29, No. 4, October 1995, pp. 41-42.
39. Holliday, M. A., "Page Table Management in Local/Remote Architectures," *Proceedings of the Second International Conference on Supercomputing,* New York: ACM Press, June 1988, p. 2.
40. Jacob, B., and T. Mudge, "Virtual Memory: Issues of Implementation," *IEEE Computer,* Vol. 31, No. 6, June 1998, pp. 37-38.
41. Dennis, J. B., "Segmentation and the Design of Multiprogrammed Computer Systems," *Journal of the ACM,* Vol. 12, No. 4, October 1965, pp. 589-602.
42. Denning, P., "Virtual Memory," *ACM Computing Surveys,* Vol. 2, No. 3, September 1970, pp. 153-189.
43. *Intel Itanium Software Developer's Manual,* Vol. 2, *System Architecture,* Rev. 2.1, October 2002, pp. 2-431-2-432.
44. Daley, R. C, and J. B. Dennis, "Virtual Memory, Processes and Sharing in Multics," *CACM,* Vol. 11. No. 5, May 1968, pp. 306-312.
45. Denning, P. X, "Third Generation Computing Systems," *ACM Computing Surveys, Vol.* 3, No. 4. December 1971, pp. 175-216.
46. Bensoussan. A.; C. T. Clingen: and R. C. Daley, "The Multics Virtual Memory: Concepts and Design," *Communications of the ACM. Vol* 15, No. 5, May 1972, pp. 308-318.
47. Organick, E. I., *The Multics System: An Examination of Its Structure.* Cambridge, MA: M.I.T Press, 1972.
48. Doran. R. W., "Virtual Memory," *Computer,* Vol. 9, No. 10, October 1976. pp. 27-37.
49. Belady, L. A.; R. P. Parmelee; and C. A. Scalzi, "The IBM History of Memory Management Technology," *IBM Journal of Research and Development,* Vol. 25, No. 5, September 1981, pp. 491-503.
50. *IA-32 Intel Architecture Software Developer's Manual,* Vol. 3, *System Programmer's Guide.* 2002, pp. 3-1-3-38.
51. da Cruz, F., "The IBM 7090," July 2003, <www.columbia.edu/ acis/history/7090.html>.
52. Harper, I, "7090/94 IBSYS Operating System," August 23,2001, <www.frobenius.com/ibsys.htm>.
53. Harper, X, "7090/94 IBSYS Operating System," August 23, 2001, <www.frobenius.com/ibsys.htm>.
54. Poulsen, L., "Computer History: IBM 360/370/3090/390," October 26, 2001, <www.beagle-ears.com/lars/engineer/comphist/ibm360.htm>.
55. Suko. R., "MVS ... a Long History." December 15, 2002, <os390-mvs.hypermart.net/mvshist.htm>.
56. Poulsen, L., "Computer History: IBM 360/370/3090/390," October 26, 2001, <www.beagle-ears.com/lars/engineer/comphist/ibm360.htm>.
57. Suko, R., "MVS ... a Long History," December 15, 2002, <os390-mvs.hypermart.net/mvshist.htm>.
58. Mealy. G., "The Functional Structure of OS/360, Part 1: Introductory Survey," *IBM Systems Journal,* Vol. 5, No. 1, 1966, <www.research.ibm.com/journal/sj/051/ibmsj0501B.pdf>.
59. Mealy, G., "The Functional Structure of OS/360, Part 1: Introductory Survey." *IBM Systems Journal,* Vol. 5, No. 1, 1966, <www.research.ibm.com/journal/sj/O5l/ibmsj0501B.pdf>.
60. Poulsen, L,, "Computer History: IBM 360/370/3090/390," October 26, 2001, <www.beagle-ears.com/lars/engineer/comphist/ibm360.htm>.
61. Suko, R., "MVS ... a Long History," December 15, 2002, <os390-mvs.hypermart.net/mvshist.htm>.
62. Poulsen, L., "Computer History: IBM 360/370/3090/390," October 26, 2001, <www.beagle-ears.com/lars/engineer/comphist/ibm360.htm>.
63. Suko, R,, "MVS ... a Long History," December 15, 2002 <os390-mvs.hypermart.net/mvshist.htm>.
64. Poulsen. L., "Computer History: IBM 360/370/3090/390," October 26, 2001, <www.beagle-ears.com/lars/engineer/comphist/ibm360.htm>.

65. Suko, R., "MVS ... a Long History," December 15,2002, <os390-mvs.hypermart.net/mvshist.htm>.
66. Poulsen, L., "Computer History: IBM 360/370/3090/390," October 26, 2001, <www.beagle-ears.com/lars/engineer/comphist/ibm360.htm>.
67. Poulsen, L., "Computer History: IBM 360/370/3090/390," October 26, 2001, <www.beagle-ears.com/lars/engineer/comphist/ibm360.htm>.
68. Suko, R., "MVS ... a Long History," December 15,2002, <os390-mvs.hypermart.net/mvshist.htm>.
69. Clark, C, "The Facilities and Evolution of MVS/ESA," *IBM Systems Journal,* Vol. 28, No. 1, 1989, <www.research.ibm.com/journal/sj/281/ibmsj28011.pdf>.
70. Poulsen, L., "Computer History: IBM 360/370/3090/390," October 26, 2001, <www.beagle-ears.com/lars/engineer/comphist/ibm360.htm>.
71. Suko, R., "MVS ... a Long History," December 15, 2002, <os390-mvs.hypermart.net/mvshist.htm>.
72. Poulsen, L., "Computer History: IBM 360/370/3090/390," October 26, 2001, <www.beagle-ears.com/lars/engineer/comphist/ibm360.htm>.
73. Suko, R., "MVS ... a Long History," December 15, 2002, <os390-mvs.hypermart.net/mvshist.htm>.
74. Suko, R., "MVS ... a Long History," December 15, 2002, <os390-mvs.hypermart.net/mvshist.htm>.
75. Clark, C, "The Facilities and Evolution of MVS/ESA," *IBM Systems Journal,* Vol. 28, No. 1, 1989, <www.research.ibm.com/journal/sj/281/ibmsj28011.pdf>.
76. Suko, R., "MVS ... a Long History," December 15,2002, <os390-mvs.hypermart.net/mvshist.htm>.
77. D. Elder-Vass, "MVS Systems Programming: Chapter 3a—MVS Internals," July 5, 1998, <www.mvsbook.fsnet.co.uk/chap03a.htm>.
78. Suko, R., "MVS ... a Long History," December 15, 2002, <os390-mvs.hypermart.net/mvshist.htm>.
79. Clark, C, "The Facilities and Evolution of MVS/ESA," *IBM Systems Journal,* Vol. 28, No. 1, 1989, <www.research.ibm.com/journal/sj/281/ibmsj28011.pdf>.
80. Suko, R., "MVS ... a Long History," December 15,2002, <os390-mvs.hypermart.net/mvshist.htm>.
81. Clark, C, "The Facilities and Evolution of MVS/ESA," *IBM Systems Journal,* Vol. 28, No. 1, 1989, <www.research.ibm.com/journal/sj/281/ibmsj2801I.pdf>.
82. Elder-Vass, D, "MVS Systems Programming: Chapter 3a—MVS Internals," 5 July 1998, <www.mvsbook.fsnet.co.uk/chap03a.htm>.
83. Suko, R., "MVS ... a Long History," December 15,2002, <os390-mvs.hypermart.net/mvshist.htm>.
84. Clark, C, "The Facilities and Evolution of MVS/ESA," *IBM Systems Journal,* Vol. 28, No. 1, 1989, <www.research.ibm.com/journal/sj/281/ibmsj28011.pdf>.
85. Suko, R., "MVS ... a Long History," December 15, 2002, <os390-mvs.hypermart.net/mvshist.htm>.
86. Clark, C, "The Facilities and Evolution of MVS/ESA," *IBM Systems Journal,* Vol. 28, No. 1, 1989, <www.research.ibm.com/journal/sj/281/ibmsj28011.pdf>.
87. Mainframes.com, "Mainframes.com—SYSPLEX," <www.mainframes.com/sysplex.html>.
88. Suko. R.."MVS ...a Long History," December 15, 2002, <os390-mvs.hypermart.net/mvshist.htm>.
89. IBM, "S/390 Parallel Enterprise Server and OS/390 Reference Guide," May 2000, <www-l.ibrn.com/servers/eserver/zseries/library/refguides/pdf/g3263070.pdf>.
90. IBM, "IBM eServer zSeries Mainframe Servers," <www-1.ibrn.com/servers/eserver/zseries/>.
91. Spruth, W.. "The Evolution of S/390," July 30, 2001, <www-ti.informatik.uni-tuebingen.de/os390/arch/history.pdf>.
92. Case, R. P. and A. Padegs, "Architecture of the IBM System/370." *Communications of the ACM,* January 1978, pp. 73-96.
93. Gifford, D. and A. Spector, "Case Study: IBM's System/360-370 Architecture," *Communications of the ACM*, April 1987, pp. 291-307.
94. IBM, "z/VM General Information, V4.4," 2003. <www.vm.ibm.com/pubs/pdf/HCSF8A60.PDF>.
95. Kutnick, D., "Whither VM?" *Datamation,* December 1, 1985. pp. 73-78.
96. Doran, R. W., "Amdahl Multiple-Domain Architecture," *Computer,* October 1988, pp. 20-28.
97. Adair, A.J.; R. U. Bayles; L. W. Comeau; and R. J. Creasy, "A Virtual Machine System for the 360/40," Cambridge, MA: *IBM Scientific Center Report 320-2007',* May 1966.
98. Lett, A.S. and W. L. Konigsford, "TSS/360: A Time-Shared Operating System," *Proceedings of the Fall Joint Computer Conference,* AFIPS, 1968, Montvale, NJ: AFIPS Press, pp. 15-28.
99. Creasy, R. J., "The Origin of the VM/370 Time-Sharing System." *IBM Journal of R&D,* September 1981, pp. 483-490.
100. IBM, "IBM z/VM and VM/ESA Home Page." <www.vm.ibm.com/>.
101. Denning, P., "Virtual Memory," *ACM Computing Surveys,* Vol. 2. No. 3, September 1970, pp. 153-189.
102. Randell, B., and C. J. Kuehner, "Dynamic Storage Allocation Systems," *Proceedings of the ACM Symposium on Operating System Principles,* January 1967, pp. 9.1-9.16.
103. Hanlon. A. G., "Content-Addressable and Associative Memory Systems—A Survey," *IEEE Transactions on Electronic Computers,* August 1966.
104. Smith, A. J., "Cache Memories," *ACM Computing Surveys,* Vol. 14, No. 3, September 1982, pp. 473-530.
105. Dennis, J. B., "Segmentation and the Design of Multiprogrammed Computer Systems," *Journal of the ACM,* Vol. 12, No. 4, October 1965, pp. 589-602.
106. Daley, R. C, and J. B. Dennis, "Virtual Memory, Processes and Sharing in Multics," *CACM,* Vol. 11, No. 5, May 1968, pp. 306-312.

107. Denning, P. J., "Third Generation Computing Systems," *ACM Computing Surveys,* Vol. 3, No. 4, December 1971, pp. 175-216.

108. Jacob, B., and T. Mudge, "Virtual Memory: Issues of Implementation," *IEEE Computer, Vol* 31, No. 6, June 1998.

109. Patterson, D., and J. Hennessy, *Computer Organization and Design,* San Francisco: Morgan Kaufmann Publishers, Inc., 1998.

no. Blaauw, G. A., and F. P. Brooks, Jr., *Computer Architecture,* Reading, MA: ACM Press, 1997.

What we anticipate seldom occurs; what we least expect generally happens.

—Benjamin Disraeli—

Time will run back and fetch the Age of Gold.

—John Milton —

Faultless to a fault.

—Robert Browning—

Condemn the fault and not the actor of it?

—William Shakespeare—

Gather up the fragments that remain, that nothing be lost.

-John 6:12-

Chapter 11

Virtual Memory Management

Objectives

After reading this chapter, you should understand:

- *the benefits and drawbacks of demand and anticipatory paging.*
- *the challenges of page replacement.*
- *several popular page-replacement strategies and how they compare to optimal page replacement.*
- *the impact of page size on virtual memory performance.*
- *program behavior under paging.*

Chapter Outline

11.1 Introduction

Chapter 9 discussed fetch, placement and replacement memory management strategies for real memory systems. Chapter 10 discussed virtual memory organization, focusing on pure paging systems, pure segmentation systems and hybridized segmentation/paging systems. In this chapter we discuss memory management in virtual memory systems.

Virtual memory fetch strategies determine when a page or segment should be moved from secondary storage to main memory. Demand fetch strategies wait for a process to reference a page or segment before loading it into main memory. Anticipatory fetch strategies use heuristics to predict which pages or segments a process will soon reference—if the likelihood of reference is high and if space is available, then the system brings the page or segment into main memory before the process explicitly references it, thus improving performance when the reference occurs.

Paging systems—either pure paging or segmentation/paging systems—that use only one page size trivialize the placement decision because an incoming page may be placed in any available page frame. Segmentation systems require placement strategies similar to those used in variable-partition multiprogramming (see Section 9.9, Variable-Partition Multiprogramming).

Replacement strategies determine which page or segment to replace to provide space for an incoming page or segment. In this chapter, we concentrate on **page-replacement strategies** that, when properly implemented, help optimize performance in paging systems. The chapter includes a discussion of Denning's Working Set Model of program behavior, which provides a framework for observing, analyzing and improving program execution in paging systems.[1]

Self Review

1. Explain the difference between demand fetch strategies and anticipatory fetch strategies in virtual memory systems. Which one requires more overhead?
2. Why are placement strategies trivial in paging systems that use only one page size?

Ans: **1)** Demand fetch strategies load pages or segments into main memory only when a process explicitly references them. Anticipatory fetch strategies attempt to predict which pages or segments a process will need and load them ahead of time. Anticipatory fetch strategies require more overhead because the system must spend time determining the likelihood that a page or segment will be referenced; as we will see, this overhead can often be small. **2)** Because any incoming page can be placed into any available page frame.

11.2 Locality

Central to most memory management strategies is the concept of locality—that a process tends to reference memory in highly localized patterns.[2,3] Locality manifests itself in both time and space. Temporal locality is locality over time. For example, if the weather is sunny in a certain town at 3 p.m., then there is a good chance (but certainly no guarantee) that the weather in that town was sunny at 2:30 p.m.

and will be sunny at 3:30 p.m. **Spatial locality** means that nearby items tend to be similar. Again, considering the weather, if it is sunny in one town, then it is likely, but not guaranteed, to be sunny in nearby towns.

Locality is also observed in operating systems environments, particularly in the area of memory management. It is an empirical (i.e., observed) property rather than a theoretical one. It is never guaranteed but is often highly likely. In paging systems, for example, we observe that processes tend to favor certain subsets of their pages, and that these pages tend to be near one another in a process's virtual address space. This behavior does not preclude the possibility that a process may make a reference to a new page in a different area of its virtual memory.

Actually, locality is quite reasonable in computer systems, when one considers the way programs are written and data is organized. Loops, functions, procedures and variables used for counting and totalling all involve temporal locality. In these cases, recently referenced memory locations are likely to be referenced again in the near future. Array traversals, sequential code execution and the tendency of programmers (or compilers) to place related variable definitions near one another all involve spatial locality—they all tend to generate clustered memory references.

Self Review

1. Does locality favor anticipatory paging or demand paging? Explain.
2. Explain how looping through an array exhibits both spatial and temporal locality.

Ans: **1)** Locality favors anticipatory paging because it indicates that the operating system should be able to predict with reasonable probability the pages that a process will use. **2)** Looping through an array exhibits spatial locality because the elements of an array are contiguous in virtual memory. It exhibits temporal locality because the elements are generally much smaller than a page. Therefore, references to two consecutive elements usually result in the same page being referenced twice within a short period of time.

11.3 Demand Paging

The simplest fetch policy implemented in virtual memory systems is **demand paging.**[4] Under this policy, when a process first executes, the system loads into main memory the page that contains its first instruction. Thereafter, the system loads a page from secondary storage to main memory only when the process explicitly references the page. There are several reasons for the appeal of this strategy. Computability results, specifically the Halting Problem, tell us that, in the general case, it is impossible to predict the path of execution a program will take (see the Operating Systems Thinking feature, Computer Theory in Operating Systems).[5-6] Therefore, any attempt to preload pages in anticipation of their use might result in the wrong pages being loaded. The overhead incurred by preloading the wrong pages can impede the performance of the entire system.

Demand paging guarantees that the system brings into main memory only those pages that processes actually need. This potentially allows more processes to

occupy main memory—the space is not "wasted" by pages that may not be referenced for some time (or ever).

Demand paging is not without its problems. A process in a demand-paged system must accumulate pages one at a time. As it references each new page, the process must wait while the system transfers that page to main memory. If the process already has many pages in main memory, then this wait time can be particularly costly, because a large portion of main memory is occupied by a process that cannot execute. This factor often affects the value of a process's **space-time product**—a measure of its execution time (i.e., how long it occupies memory) multiplied by the amount of space in main memory the process occupies (see the Operating Systems Thinking feature, Space-Time Trade-offs).[7] The space-time product illustrates not only how much time a process spends waiting, but how much main memory cannot be used while it waits. Figure 11.1 illustrates the concept. The y-axis represents the number of page frames allocated to a process and the *x*-axis represents "wall clock" time. The space-time product corresponds to the area under the "curve" in the figure; the dotted lines indicate that a process has referenced pages that are not in main memory and must be loaded from the backing store. The shaded region repre-

Operating Systems Thinking

Computer Theory in Operating Systems

Computer science is rich in elegant theoretical fields. Computability theory helps us determine what kinds of things computer software, such as operating systems, can and cannot do. The Halting Problem of computability theory tells us that in the general case, we cannot write a program to determine the execution path of another program. This has many ramifications in operating systems. If we could predict the execution path of a program, then we could for example, implement perfect anticipatory resource allocation, so that the resources are ready (if at all possible) when the process needs them, avoiding lengthy delays. Once we know what computers can and cannot do, complexity theory helps us determine whether those tasks can be performed efficiently and to characterize just how efficiently (or inefficiently) they can be performed. Automata theory helps us understand the power of different classes of computing devices; as we discussed in Chapter 6, this helps us realize that modern computer systems are far too complex to allow exhaustive testing of all possible states of computer hardware and software. This last observation is truly a point to ponder, especially if you must build highly reliable systems.

sents the process's space-time product while it is performing productive work. The unshaded region represents its space-time product while it waits for pages to be loaded from secondary storage. *[Note:* The wait period, *F,* is much larger than indicated by the figure.] Thus, the unshaded region indicates the amount of time during which the process's memory allocation cannot be used. Reducing the space-time product of a process's page waits, in order to improve memory utilization, is an important goal of memory management strategies. As the average page-wait time increases, the benefit of demand paging decreases.[8]

Self Review

1. Why is the space-time product of demand paging higher than that of anticipatory paging?
2. How could demand paging (as compared to anticipatory paging) increase the degree of multiprogramming in a system? How could demand paging decrease the degree of multiprogramming in a system?

Ans: **1)** The reason is that the process has pages in memory that it is not using while it waits for its pages to be painstakingly demand-paged in, one at a time. Anticipatory paging also has a space-time product with associated waste. Pages brought into memory before they are referenced occupy page frames that go unused, thus preventing other processes from using them. As

Operating Systems Thinking

Space-Time Trade-offs

Examples of the space-time trade-off are common in computing and other areas. When moving to a new apartment, if you have a larger truck, you can complete the move in less time, but you will have to pay more to rent the larger truck. When you study searching and sorting algorithms in data structures and algorithms classes, you see how performance can improve when more memory is available (e.g., hash tables). These trade-offs are common in operating systems as well. If the operating system allocates more main memory to an executing program, for example, the program may run significantly faster. With less, more-expensive memory, the operating system must manage the memory more extensively, incurring more processing overhead. With more abundant, cheaper memory, the operating system can manage the memory less intensively, perhaps making cruder decisions while consuming less processor power. We will see in the discussion of RAID systems in Chapter 12, that maintaining redundant copies of data can result in improved system throughput. We will see, however that increasing the amount of memory available to a process does not always increase the speed at which it runs—we will study Belady's Anomaly which shows that in some circumstances, giving a process more memory could actually degrade performance. Fortunately, this occurs only in rare circumstances.

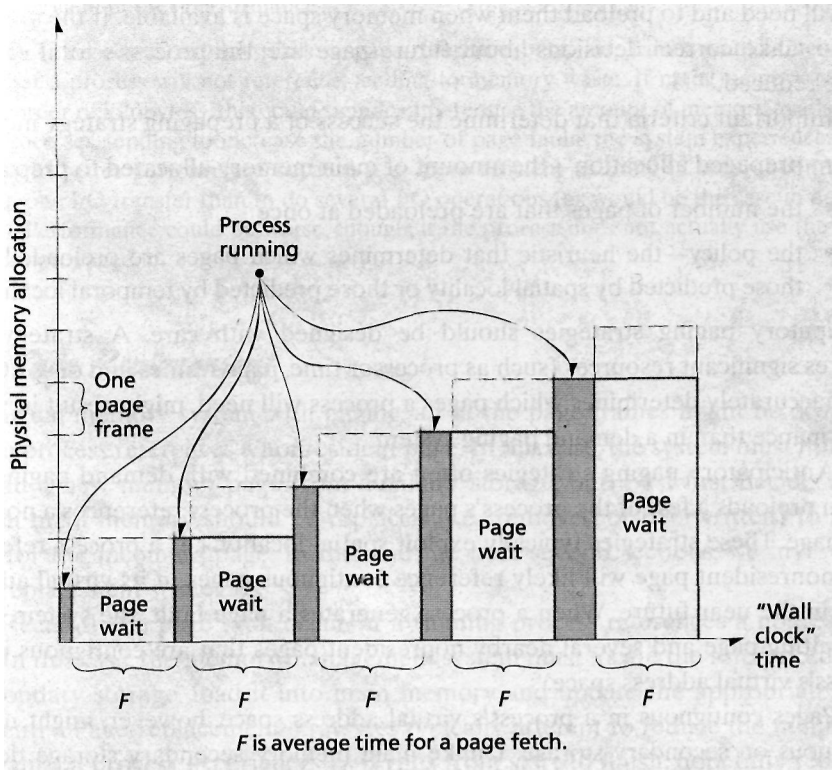


Figure 11.1 | *Space-time product under demand paging.*

we will see, anticipatory paging brings these pages into main memory in groups, which generally reduces the page-wait time compared to loading pages individually using demand paging. **2)** Demand paging could increase the degree of multiprogramming because the system brings into main memory only those pages that processes actually needed. Therefore, more processes can fit into physical memory. However, processes require more execution time because they need to retrieve pages from secondary storage more often (compared with being brought in as a group with anticipatory paging). While the operating system is retrieving pages from secondary storage, the memory that the process takes up is wasted, and thus the degree of multiprogramming might decrease (over what is possible with anticipatory paging).

11.4 Anticipatory Paging

A central theme in resource management is that a resource's relative value influences just how intensively the resource should be managed.[9] As hardware costs continue their dramatic decline, the relative value of machine time to people time is reduced. Operating systems designers are constantly focused on reducing the amount of time people must wait for results from computers.[10]

As we demonstrated in the preceding section, one way to reduce wait times is to avoid the delays in a demand-paged system. In **anticipatory paging** (also called

prefetching and **prepaging),** the operating system tries to predict the pages a process will need and to preload them when memory space is available. If the system is able to make correct decisions about future page use, the process's total runtime can be reduced.[11]-[12]

Important criteria that determine the success of a prepaging strategy include:

- prepaged allocation—the amount of main memory allocated to prepaging
- the number of pages that are preloaded at once
- the policy—the heuristic that determines which pages are preloaded (e.g., those predicted by spatial locality or those predicted by temporal locality).[13]

Anticipatory paging strategies should be designed with care. A strategy that requires significant resources (such as processor time, page frames and disk I/O), or that inaccurately determines which pages a process will need, might result in worse performance than in a demand paging system.

Anticipatory paging strategies often are combined with demand paging; the system preloads a few of the process's pages when the process references a nonresident page. These strategies typically exploit spatial locality; i.e., a process referencing a nonresident page will likely reference contiguous pages in its virtual address space in the near future. When a process generates a page fault, the system loads the faulting page and several nearby nonresident pages that are contiguous in the process's virtual address space.

Pages contiguous in a process's virtual address space, however, might not be contiguous on secondary storage. Unlike main memory, secondary storage devices (e.g., hard disks) do not provide uniform access times to data stored at different locations, so the time required to load multiple pages from secondary storage may be significantly greater than that required to load a single page. In this case, processes could suffer greater page-wait times due to anticipatory paging than in demand paging.

One solution is to group pages on secondary storage that are contiguous in a process's virtual address space.[14] As we discuss in Chapter 12, Disk Performance Optimization, the difference between the page-wait times for loading several pages that are contiguous on disk and for loading a single page is relatively small, so anticipatory paging can be performed without significantly increasing the page-wait time compared to demand paging. In Linux, for example, when a process references a nonresident page containing its program instructions or data, the kernel attempts to exploit spatial locality by loading from disk the nonresident page and a small number of pages that are contiguous in the process's address space. By default, the system loads from disk four contiguous pages (16KB on the IA-32 architecture) if main memory is smaller than 16MB, or eight contiguous pages (32KB on the IA-32 architecture) otherwise.[15] This technique can yield good performance for processes that exhibit spatial locality.

Self Review

1. In which scenarios is the Linux anticipatory paging strategy inappropriate?

2. Why is anticipatory paging likely to yield better performance than demand paging? How might it yield poorer performance?

Ans: **1)** If processes exhibit random page-reference behavior, Linux would likely load pages that a process will not reference, leading to memory waste. If main memory is small (on the order of kilobytes), this could significantly reduce the amount of memory available to other processes, tending to increase the number of page faults the system experiences. **2)** It could yield better performance because it is more efficient to bring in several contiguous pages in one I/O transfer than to do several I/O operations (as would be the case in demand paging). Performance could be worse, though, if the process does not actually use the pages that were prepaged in.

11.5 Page Replacement

In a virtual memory system with paging, all of the page frames might be occupied when a process references a nonresident page. In this case, the system must not only bring in a new memory page from auxiliary storage, but must first decide which page in main memory should be replaced (i.e., removed or overwritten) to make room for the incoming page. In this and the next several sections, we investigate page-replacement strategies.

Recall that a page fault occurs if a running process references a nonresident page. In this case, the memory management system must locate the referenced page in secondary storage, load it into main memory and update the appropriate page table entry. Page-replacement strategies typically attempt to reduce the number of page faults a process experiences as it runs from start to finish, hopefully reducing the process's execution time.

If the page chosen for replacement has not been modified since it was last paged in from disk, then the new page can simply overwrite it. If the page has been modified, it must first be written (or evicted) to secondary storage to preserve its contents. A **modified bit,** or **dirty bit,** in the page table entry is set to 1 if the page has been modified and 0 otherwise.

Writing (or **flushing)** a modified page to disk, which requires a disk I/O operation, increases page-wait times if it is performed when a page is replaced. Some operating systems, such as Linux and Windows XP, periodically flush dirty pages to secondary storage to increase the likelihood that the operating system can perform page replacement without first having to write a modified page to disk. Because this periodic flushing can occur asynchronously with process execution, the system incurs little overhead by doing so. If a process references a modified page before the flush is completed, then it is recaptured, thus saving an expensive page-in operation from secondary storage.

When we evaluate a page-replacement strategy, we often compare it to the so-called **optimal page replacement** strategy (also called **OPT** or **MIN),** which states that, to obtain optimum performance, replace the page that will not be referenced again until furthest in the future.[16, 17, 18, 19, 20] Thus, the strategy increases performance by minimizing the number of page faults. This strategy can be demonstrated